România
Ministerul Apărării Naționale
Academia Tehnică Militară "*Ferdinand I*"

Facultatea de Sisteme Informatice și Securitate Cibernetică
CALCULATOARE ŞI SISTEME INFORMATICE PENTRU APĂRARE ŞI SECURITATE NAŢIONALĂ

Platformă de Analiză Automată a Atacurilor Data Poisoning asupra unei Infrastructuri de Învățare Federate

Coordonator Științific
Cpt. conf. dr. ing. Iulian Aciobăniței

Absolvent
Sd. Sg. Maj. Lepădatu Tudor

Conține _____ file
Inventariat sub numărul _____
Cu poziția din indicator _____
Cu termen de păstrare _____

București
An 2026

Mulțumiri pentru persoanele care au sprijinit procesul de realizare a acestei lucrări

Referatul Coordonatorului Științific

Referatul reprezintă un text în care coordonatorul științific sumarizează, ulterior finalizării conținutului efectiv, efortul pe care l-ați depus și trage concluzii cu privire la gradul de realizare a obiectivelor propuse inițial (cele prezentate în cadrul detalierii). De regulă, acest referat nu depășește cele 4 pagini alocate în acest document.

Declarație de originalitate

Subsemnatul/Subsemnata, **Sd. Sg. Maj. Lepădatu Tudor**, absolvent(ă) al/a **Academia Tehnică Militară "*Ferdinand I*"**, **Facultatea de Sisteme Informatice și Securitate Cibernetică**, declar pe propria răspundere că lucrarea de diplomă cu titlul " **Platformă de Analiză Automată a Atacurilor Data Poisoning asupra unei Infrastructuri de Învățare Federate** " este rezultatul activității mele independente, nu a fost plagiată sau copiată din alte surse și respectă normele de etică academică.

Semnătura,

.....

Tema Proiectului de Diplomă

Tema proiectului de diplomă (sau detalierea) reprezintă un text realizat de coordonatorul științific, în lunile următoare propunerii titlului proiectului de diplomă către facultate, în care detaliază nevoia reală a implementării unui astfel de proiect și modul în care se dorește a fi implementat. În plus, poate prezenta structura pe capitole a viitoarei lucrări, anexele ce vor fi incluse și sursele bibliografice din care studentul se va informa. De regulă, acestă detaliere nu depășește cele 4 pagini alocate în acest document.

Abstract

The Artificial Intelligence integration with tools and applications has evolved since 2020s and the cybersecurity scene tries to adapt frequently. Information security and data integrity is more important than never before, being used by Machine Learning models or Neural Networks trained to perform specific tasks.

From a data science point of view, the quality of information is much important than securing it. The AI evolution has led to the creation of different attack boundaries, from changing the model parameters to perform data poisoning schemes. Confidentiality is the key in maintaining unique aspects for each entity involved in the federated learning process.

In this paper, data poisoning attacks are studied with different options in a segregated simulated infrastructure called federated learning, in which each client may change its scope intentionally or unintentionally. Each simulation has its own configuration providing data scientist with a dedicated environment for testing its machine learning algorithm againts data poisoning attacks.

Cuprins

1 Introducere **1**
1.1 Context . . . 1
1.2 Motivatia lucrarii . . . 2
1.3 Obiectivele lucrarii . . . 2
1.4 Structura lucrarii . . . 3

2 Notiuni Teoretice **4**
2.1 Notiuni introductive . . . 4
2.1.1 Diferenta dintre Machine Learning si Deep Learning . . . 4
2.1.2 Retea Neuronala . . . 5
2.2 Invatare automata federata . . . 5
2.2.1 Concept . . . 5
2.2.2 Arhitectura FL . . . 5
2.2.3 Procesul de antrenare FL . . . 8
2.2.4 Exemple in viata reala . . . 9
2.3 Atacuri de tip Data Poisoning . . . 10
2.3.1 Definirea tipurilor de atac . . . 10
2.3.2 Vectori de atac . . . 11
2.3.3 Atacul Data poisoning . . . 12
2.3.4 Impactul poisoning in Federated Learning . . . 12
2.4 Alte Notiuni . . . 13
2.4.1 Docker . . . 13
2.4.2 Python . . . 14
2.4.3 Rest API . . . 15

3 Proiectare, Implementare si Testare **17**
3.1 Arhitectura Platformei . . . 17
3.2 Implementare Platformei . . . 21
3.3 Cerintele Software . . . 24
3.3.1 Cerintele functionale . . . 24
3.3.2 Cerintele nefunctionale . . . 28
3.4 Testare . . . 30
3.4.1 Testarea de Validare a Șabloanelor . . . 30
3.4.2 Testarea Pipeline-ului de Simulare . . . 30
3.4.3 Testarea de Integrare . . . 31
3.4.4 Testarea Comunicării WebSocket . . . 31
3.4.5 Testarea Managementului GPU . . . 31
3.4.6 Testarea Fluxului de Anulare . . . 31
3.4.7 Testarea de Securitate . . . 32
3.5 Modele de Analiză . . . 32
3.5.1 Diagrama de Context . . . 32
3.5.2 Diagrama de Componente . . . 33
3.5.3 Diagramele de Flux . . . 34
3.5.4 Diagrama Entitate-Relație (ERD) . . . 36
3.5.5 Diagrama Cazurilor de Utilizare . . . 37

4 Rezultate și Metrici Simulări **39**
4.1 Studiu de Caz 1: Creșterea intensității otrăvirii datelor 39
4.1.1 Detalii practice 39
4.1.2 Rezultate experimentale 39
4.2 Studiu de Caz 2: Cea mai bună protecție pentru un atac specific 40
4.2.1 Detalii practice 40
4.2.2 Rezultate experimentale 40

5 Concluzii si dezvoltare ulterioara **43**
5.1 Starea Curenta 43
5.2 Dezvoltare Ulterioara 43

Bibliografie **44**

Listă de figuri

2.1 Arhitectura interna a unui dispozitiv [7] 8
2.2 Procedee in invatare automata federata [7] 9

3.1 Medii de python platforma 18
3.2 Arhitectura si fluxul platformei 25
3.3 Diagrama de Context — Nivel 0: Marginile Platformei 32
3.4 Diagrama de Context — Nivel 1: Fluxul de date 33
3.5 Diagrama de Componente a Platformei 34
3.6 Diagrama Flux Utilizator 35
3.7 Diagrama Fluxului de Simulare 36
3.8 Diagrama Entitate-Relație (ERD) 37
3.9 Diagrama Cazurilor de Utilizare 38

4.1 Rezultate Experimentale - Cazul 1 40
4.2 Rezultate Experimentale - Cazul 2 42

Listă de Abrevieri

UE . . . . . Uniunea Europeană

EU . . . . . *European Union*

Capitolul 1:

Introducere

1.1 Context

Odata cu dezvoltarea sistemelor de calcul moderne si a componentelor Hardware, s-au putut realiza produse software complexe cu capacitati de stocare net superioare. Revolutia tehnologica a permis nu doar realizarea unor sarcini simple, precum calcule matematice, sau automatizarea unor dispozitive (de exemplu aprinderea automata a unui bec printr-un microcontroler), ci si posibilitatea gestionarii mai eficiente a informatiilor digitale (de la date bancare la fisiere media).

Aceasta a devenit treptat principala sursa legitima de inregistrare a oricarui tip de date (text, imagini, video, audio). Pentru a accesa si actualiza informatia digitala, s-au dezvoltat diferite versiuni de baze de date centralizate si distribuite.

Bazele de date centralizate sunt aplicatii software specializate ce folosesc resursele sistemului (a statiei) pentru a raspunde cat mai rapid interogarilor. Statiile trebuie sa detina multa putere de stocare si de procesare in comparatie cu un sistem de calcul normal destinat utilizatorilor casnici. In alta ordine de idei, s-au dezvoltat si baze de date distribuite, menite sa reduca din capacitatile tehnice ale serverului si sa stocheze informatia sub forma descentralizata. Cautarea resursei in acest context ar presupune interogarea recursiva a fiecarei entitati pana la gasirea sa. Prin acest mod, nu doar ca statiile pot avea si capabilitati tehnice mai reduse, dar s i pot pastra copii de rezerva (backup) locale pentru fiecare segment de informatie in parte.

Această evoluție naturală spre descentralizare a deschis drumul unor concepte moderne precum învățarea automată federată (federated learning), unde datele nu mai sunt transferate către un server central. În schimb, modelele sunt antrenate local, iar parametrii sunt ulterior agregați global. Astfel, se menține confidențialitatea datelor, fără a compromite performanța modelului.

Evolutia tehnologica continua a dat nastere la o serie de atacuri cibernetice menite sa destabilizeze securitatea aplicatiilor si totodata sustragerea a cat mai multe date sau identitati private in contradictie cu normele legale. Cele mai populare atacuri raportate la scara globala pentru anul curent 2025 sunt ransomware (conform [1], in SUA s-au raportat cresteri de 149%), furtul de identitate prin exfiltrarea de credentiale, si phishing. Dezvoltarea modelelor de inteligență artificială a amplificat aceste riscuri, oferind atacatorilor instrumente automatizate pentru generarea și adaptarea atacurilor.

Pentru a limita utilizarea abuzivă a tehnologiilor bazate pe AI, Uniunea Europeană a adoptat în 2024 un set de reglementări stricte privind integrarea acestor module în aplicațiile software, prin AI Act [2].

Progresul din domeniul machine learning si a Large Language Models a fost posibil ca urmare a unui volum masiv de date disponibile si a nevoii tot mai mari de analiza. Acest lucru a determinat aparitia unei noi categorii de specialisti, data scientists, dedicati colectarii si prelucrarii minutioase a datelor pentru antrenarea modelelor.

Totuși, pe măsură ce investițiile în tehnologii AI au crescut, au apărut și actori rău intenționați care încearcă să exploateze vulnerabilitățile din procesul de antrenare. Întrucât modelele moderne depind de calitatea datelor folosite, acestea au devenit o țintă principală a atacurilor. Atacatorii se regasesc si ei intr-o pozitie constanta de adaptare la noile formalitati de securitate si incearca sa contracareze fiecare element nou. Astfel, avand in vedere complexitatea dezvoltarii

unui modul de inteligenta artificiala specializat pe diferite domenii, tinta s-a redirectionat spre volumul de date pe care acestea le folosesc si care pot determina starea finala a aplicatiei.

În contextul învățării automate distribuite, literatura de specialitate identifică trei categorii majore de atacuri:

- Atacuri asupra datelor, precum data poisoning, unde setul de antrenare este manipulat pentru a altera comportamentul modelului;

- Atacuri asupra modelului, prin modificarea parametrilor sau a gradientului (de exemplu, model poisoning);

- Atacuri asupra canalului de comunicare, care vizează interceptarea sau modificarea mesajelor dintre entitățile participante.

Lucrarea de față se concentrează pe prima categorie, data poisoning, în cadrul unei infrastructuri de invatare federate.

1.2 Motivatia lucrarii

Avand in vedere aspectele legate de posibilitatea unei interventii asupra setului de date de antrenare, atac denumit otravire a datelor (data poisoning), munca cercetatorilor s-a ingreunat. Preocuparea nu mai este primordial asupra analizei setului de date de antrenare, cat despre mentinerea integritatii si a confidentialitatii lor. Pentru a răspunde acestor nevoi, colaborarea dintre cercetători s-a orientat către modele distribuite de lucru, iar învățarea federată (federated learning) a devenit una dintre principalele direcții. Aceasta permite colaborarea între participanți fără a partaja direct seturile lor de date, menținând o barieră naturală împotriva accesului neautorizat. Totuși, deși infrastructura este diferită față de abordările centralizate, vulnerabilitățile rămân, iar atacurile asupra datelor utilizate local pot afecta modelul global.

In urma unei analize proprii, am putut observa diferite solutii/frameworks de simulare a procesului de invatare automata federata, dar fara o integrare cu mecanisme moderne de testare pentru atacuri precum otravirea datelor (data poisoning) amintite anterior [3]. Unele dintre aceste framework-uri sunt poate dificil de gestionat si configurat [4], si nu permit extinderea usoara prin integrare altor componente. In acelasi timp, gandindu-ne la multitudinea de atacuri malware si la platformele de detectie a lor, devine clar ca in domeniul inteligentei artificiale lipseste o platforma centralizata, flexibila, dedicata testarii si evaluarii cu diferite tipuri de atacuri asupra modelelor distribuite.

Aceste limitări justifica realizarea prezentei lucrari, care își propune dezvoltarea unei platforme de simulare capabile sa testeze atacuri de tip data poisoning într-o infrastructura de învățare federată.

1.3 Obiectivele lucrarii

Plecand de la neajunsurile prezentate, ne propunem in aceasta lucrare sa venim in sprijinul comunitatii de cercetare stiintifica in domeniul securizarii solutiilor cu AI cu o platforma de simulare cu sursa deschisa ("open source"), a acestei clase de atacuri pe mai multe directii. Astfel, oferim cercetatorilor posibilitatea analizei algoritmului de antrenare propriu dezvoltat, plecand de la o retea neuronala de baza si un set de date uzual (imagini), si testarea sa prin antrenare in diferite conditii. Platforma in sine respecta toate normele unei aplicatii software de productie, in care fiecare actiune are propria sa logica de implementare. Serviciile sunt segregate suficient de mult incat sa permita o dezvoltare ulterioara prin integrarea lor cu alte sisteme.

Rezultatele pot fi utile in contextul securizarii procesului de antrenare al algoritmului, dar si pentru analiza factorilor de risc la care e expus in acest mediu.

Cercetatorul este cel care furnizeaza algoritmul python de antrenare a propriei retele neuronale sau algoritm de Machine Learning. El seteaza parametrii simularii atat pentru procesul de antrenare, cat si pentru tipul de atac de otravire a datelor. Platforma isi propune sa simuleze acest tip de atac cu ajutorul acestor setari de inceput intr-un mediu de invatare federata, furnizand la final o comparatie intre modelul antrenat folosind datele normale de antrenare si cel antrenat cu datele otravite. Aceste rezultate pot fi utile in semnalarea unui posibil risc la nivelul modelului dezvoltat, oferind mai apoi solutii de imbunatatire a implementarii sale.

1.4 Structura lucrarii

Capitolul 2:

Notiuni Teoretice

In acest capitol, vor fi prezentate notiunile teoretice specifice intelegerii procesului de dezvoltare a platformei de simulare. Vom incepe cu Notiunile introductive despre conceptele de Machine Learning in antiteza cu Deep Learning. In continuare, vom discuta despre invatarea federata si arhitectura unei infrastructuri federate de invatare automata, tipurile de atacuri data poisoning implementate in procesul de simulare a atacurilor, precum si alte notiuni specifice implementarii.

2.1 Notiuni introductive

Machine Learning si Deep Learning sunt doua ramuri importante ale Inteligentei Artificiale care au rolul dezvoltarii unor modele specifice rezolvarii unor anumite actiuni. Pornind de la antrenarea de retele neuronale, ne orientam atentia spre setul de date de antrenare si spre actorii ce pot interveni in acest proces. Mediul in care testam ofera o perspectiva reala asupra impactului pe care il pot avea aceste atacuri la nivelul unei organizatii sau aplicatii.

2.1.1 Diferenta dintre Machine Learning si Deep Learning

Inteligența Artificială (AI) este domeniul vast care înglobează orice tehnică ce permite calculatoarelor să imite comportamentul uman. Informatia a evoluat treptat odata cu imbunatatirea capabilitatilor de stocare ale dispozitivelor si aparitia programelor software complexe. De la simplul format de text, inregistrari audio, pana la imagini si video in rezolutii 4K, modul de lucru s-a diversificat constant.

La fel au evoluat si cerintele utilizatorilor, care tind sa acceada catre solutii automate care sa le rezolve problemele uzuale, precum identificarea de patterns in imagini sau chiar din video, sau generarea de text.

IA vine sa rezolve aceste probleme si sa introduca algoritmi de rezolvare specifici pentru fiecare tip de informatie furnizata.

Machine Learning este o componenta importanta din domeniul IA care se diferentiaza de alte metode de antrenare prin optimizarile pe care le aduce erorilor ce apar din predictia rezultatului corect. Modelele de ML clasice se bazeaza pe interventia umana in factorul de decizie (supervised learning), mai precis datele de intrare sunt etichetate pentru a oferi un context de predictie stabil.

Deep Learning este o subcategorie a Machine Learning, care are rolul de a minimiza interventia umana si a automatiza procesul de decizie. Prin aceasta metoda se automatizeaza mare parte din extragerea caracteristicilor pe setul de date, eliminand nevoia de a defini etichete pentru fiecare valoare de intrare (unsupervied learning).

Diferenta dintre aceste doua concepte este in modul in care acesti algoritmi invata si procentul de utilizare a datelor [5]. Scopul principal al invatarii automate este predictia. Pe baza unui set de date de antrenare si de testare, se determina o anumita categorie de iesire predefinita.

2.1.2 Retea Neuronala

Retelele Neuronale sunt un subset al Machine Learning si se identifica drept infrastructura de baza din cadrul algoritmilor de Deep Learning. Denumirea de "neuronal" se refera la structura lor interna, in care fiecare caracteristica (feature) este un neuron ce interactioneaza unii cu altii. Ele sunt compuse din 3 straturi/layers: primul strat il reprezinta stratul nodurilor de intrare, al doilea strat este denumit "stratul ascuns" (hidden layer) pt ca incapsuleaza mai multe straturi, iar ultimul strat este cel de iesire in care se face predictia propriu-zisa. Straturile ascunse sunt concepute pentru a procesa iterativ datele pornind de la starea lor din nodurile de intrare pana la stratul de iesire.

2.2 Invatare automata federata

Evolutia hardware in tehnologie a condus la cresterea numarului dispozitivelor mobile (telefoane, tablete), denumite gadgets din faptul ca sunt mici, portabile si moderne. Ele au fost mai departe adoptate la scara larga, devenind obiecte indispensabile in era tehnologica ce avea sa vina.

2.2.1 Concept

Invatarea automata federata permite lucrul cu modele de ML sau chiar retele neuronale, antrenate distribuit, pe un numar mare de dispozitive in scopul rezolvarii unei probleme de IA. Distribuirea sarcinilor a fost adoptata si in contextul opozitiei lucrului centralizat, pe servere ce detin capabilitati Hardware performante (placi grafice de ultima generatie), dar care genereaza costuri mari si care pot fi predispuse la amenintari de securitate cibernetica, fiind considerate SPOF(Single Point of Failure).

2.2.2 Arhitectura FL

In literatura, exista mai multe categorii de arhitecturi de invatare automata federata. In aceasta sectiune ne vom concentra pe clasificarea generala a arhitecturii unei aplicatii folosind federated learning, si vom enumera pentru o anumita categorie cum se clasifica dispozitivele utilizate.

Federated Learning, asa cum a mai fost mentionat, este organizat dintr-un server (agregator) si multiple dispozitive client. Modul in care aceste entitati comunica este fundamentul principal in modului de imbunatatire al invatarii.

In modul clasic al federated learning, dispozitivele client transmit actualizari ale modelului de baza la un server central care aplica asupra lor o functie de agregare, reconstruind intreg modelul de baza. Aceasta setare/model, presupune de fapt o delegare a sarcinii de invatare, insa pastreaza entitate centrala necesara imbunatatirii solutiei. Acest fapt, nu tine sa evita posibilitatea amenintarilor cibernetice (Single Point of Failure), ci doar sa usureze costurile centralizatorului in a procesa local problema, distribuind sarcinile.

Modelul Fully decentralized (peer-to-peer) learning, ofera o noua abordare si rezolva problema cibernetica amintita. In aceasta setare nu exista agregator, imbunatatirile fiind comunicate intre clienti interconectati. Ideea principala se bazeaza pe inlocuirea comunicarii cu agregatorul cu cea intre dispozitive individuale printr-un protocol prestabilit. In functie de numarul de dispozitive, se concepe un graf de conexiuni in care fiecare nod reprezinta un client, iar fiecare muchie un canal de comunicatie. Restrictia principala este ca un dispozitiv sa fie conectat la un numar maxim limitat de dispozitive adiacente, prestabilit, in contradictie cu un graf complet

(stea) specific arhitecturii clasice client-server. Nodurile isi imbunatatesc propriile variante ale retelei, si isi comunica rezultatele pe care le agrega local, realizand o medie a ponderilor. In comparatie cu modelul federated learning clasic, modelul fully decentralized nu specifica de la inceput dispozitivelor un model de baza global de la care sa porneasca in procesul de rezolvare a problemei.

	Federated Learning	**Fully Decentralized (peer-to-peer) Learning**
Orchestrare	Un server central de orchestrare organizează antrenarea, dar nu vede niciodată datele brute.	Nu există orchestrare centralizată.
Comunicare pe arie largă	De obicei topologie hub-and-spoke, în care hub-ul reprezintă un furnizor de servicii coordonator (de obicei fără date), iar spoke-urile conectează clienții.	Topologie peer-to-peer, cu un graf de conectivitate posibil dinamic.

Tabel 2.1: Comparație între modelele Federated Learning [6]

Imaginea de mai sus ofera o privire de ansamblu asupra celor doua modele de arhitecturi si caracteristicile acestora.

Model	Avantaje	Dezavantaje
Federated learning (centralized coordination)	• mai simplu de configurat (topologie hub-and-spoke) • pornește de la un model global de bază • agregarea centralizată reduce sarcina de calcul pe clienți • necesită mai puține conexiuni (doar client → server) • gestiunea și monitorizarea sunt mai simple	• SPOF (Single Point of Failure) - serverul central • serverul poate deveni țintă pentru atacuri • nu elimină riscurile cibernetice, doar distribuie munca • dependența de coordonator pentru progresul antrenării • necesită infrastructură centralizată permanent disponibilă
Fully decentralized / peer-to-peer learning	• previne atacurile specifice unui server central (evită SPOF) • reziliență crescută - compromiterea unui nod nu destabilizează întregul sistem • îmbunătățirile se propagă prin graf, fără entitate centrală • agregare locală (fiecare nod mediază ponderile) • poate scala natural dacă graful este bine proiectat	• necesită conexiuni suplimentare între clienți • topologie complexă, dificil de administrat • nu există model global inițial furnizat tuturor • performanța depinde de calitatea grafului de conexiuni • nodurile malițioase pot influența direct vecinii • necesită protocoale suplimentare pentru consistența actualizărilor

Tabel 2.2: Compararea modelelor Federated Learning și Fully Decentralized Learning

In tabelul de mai sus, sunt evidentiate avantajele si dezavantajele utilizarii celor doua tipuri de modele federated learning.

In continuarea acestei lucrari, se va discuta preponderent despre modelul clasic federated learning, fiind unul adoptat la scara larga si care ofera performante bune raportat la costurile de productie.

Modelul clasic la randul sau, se catalogheaza in literatura in functie de tipul dispozitivelor care iau parte la procesul de antrenare. Sub acest filtru, exista Cross-Device Federated Learning si Cross-Silo Federated Learning.

Cross-Device Federated Learning sunt dispozitivele client IOT uzuale, individuale, care comunica orchestratorului printr-un protocol prestabilit.

Cross-Silo Federated Learning sunt dispozitive din institutii guvernamentale, companii, sau centre de date distribuite geografic. Institutiile nu doresc sa schimbe informatii intre ele sau cu un furnizor de servicii central, pastrandu-si confidentialitatea, folosind federated learning pt a antrena propriul model pe datele private ale fiecaruia.

2.2.3 Procesul de antrenare FL

Primul pas este stabilirea conexiunii dintre dispozitive si un server de agregare ce permite antrenare distribuita a tipului de retea neuronala sau model ML specific problemei. Odata stabilit canalul, in faza de configurare initiala, serverul trimite dispozitivelor starea de baza a retelei neuronale, ponderile, in vederea antrenarii individuale. Fiecare retea se antreneaza cu datele extrase local (on device) si isi imbunatateste configuratia interna la fiecare epoca pentru o perioada de timp bine determinata.

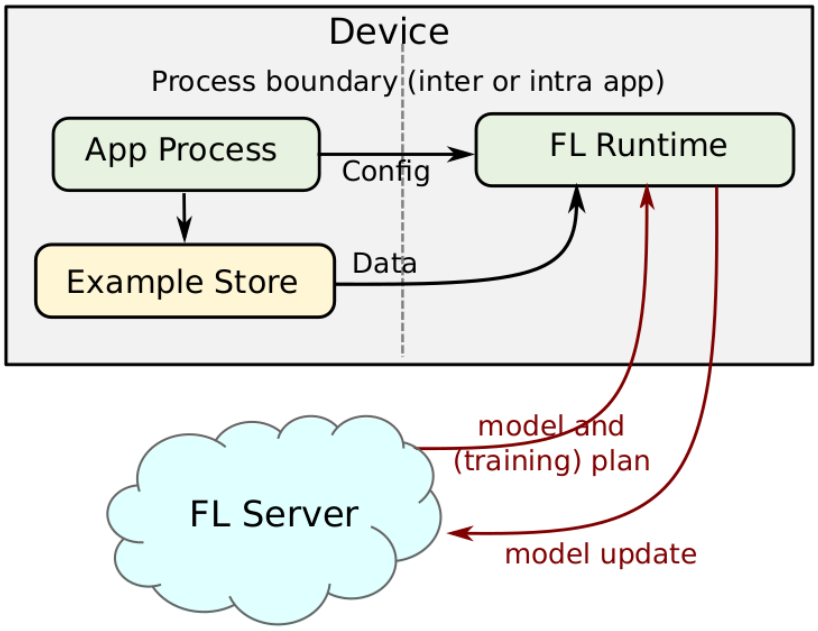

Figura 2.1: Arhitectura interna a unui dispozitiv [7]

Figura de mai sus descrie operatiile specifice programului Software care se ocupa de antrenarea retelei/modelului. Putem observa cum dispozitivele primesc un plan de antrenare de baza pe care il vor antrena local pe un set de date limitat.

Desi acest pas nu aduce un procent de imbunatatiri foarte mari, in faza urmatoare, dispozitivele vor transmite configuratiile curente ale retelelor lor la orchestrator (server). Rutina

FL_Runtime extrage configuratia noua locala si ii comunica serverului pentru o posibila actualizare a sa.

In cele din urma, entitatea centrala combina toate aceste ponderi aplicand o functie de agregare si in cazul imbunatatirii setului de ponderi, modifica configuratia de baza si o retransmite dispozitivelor pereche. Daca ponderile noi nu se imbunatatesc semnificativ fata de configuratia de baza, atunci se patreaza aceasta din urma, iar in caz contrar se actualizeaza cu noile ponderi.

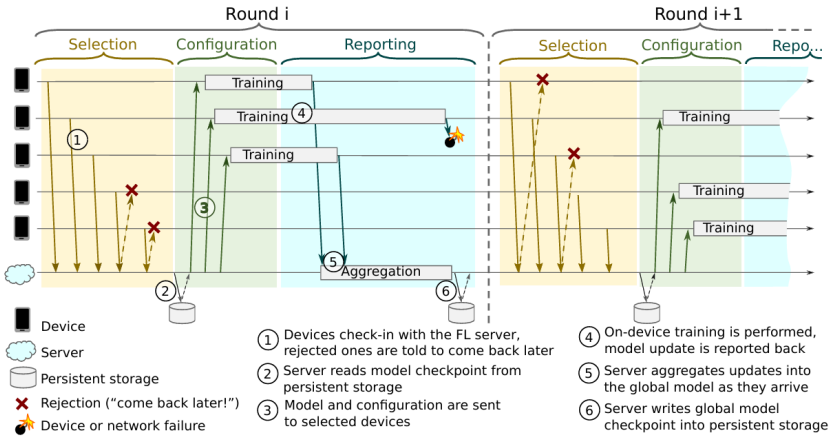

Figura 2.2: Procedee in invatare automata federata [7]

In figura de mai sus, se pot observa intr-o maniera continua, fluxul de comunicatie dintre dispozitive si serverul agregator, precum si operatiile specifice fiecarei entitati dintr-o runda de mesaje.

Securitatea protocoalelor de agregare, utilizate in comunicatii dintre clienti si orchestrator, este o componenta importanta in procesul federated learning. De mentionat este faptul ca, in aceasta topologie, comunicatiile au loc criptat, folosind metode specifice precum criptare homomorfica, sau chiar OTP, insa securitatea datelor de pe dispozitive ramane la latitudinea acestuia.

2.2.4 Exemple in viata reala

Federated Learning s-a extins rapid în numeroase domenii datorită capacității sale de a antrena modele performante fără a colecta sau centraliza date sensibile. Prin păstrarea informațiilor la nivelul fiecărui dispozitiv sau instituții, FL reduce riscurile asociate scurgerilor de date și permite colaborarea între entități care altfel nu ar putea împărtăși date brute. În continuare sunt prezentate câteva exemple reprezentative ale utilizării sale în aplicații din lumea reală.

Industrie și IoT

- **Mentenanță predictivă:** Vehiculele moderne, utilajele industriale și echipamentele IoT generează constant date despre starea componentelor. FL permite antrenarea unui model comun care poate prezice momentul oportun pentru realizarea mentenanței fără a colecta date brute de la fiecare dispozitiv.

- **Dispozitive de monitorizare:** Senzori purtabili și dispozitive smart home pot furniza statistici privind activitatea sau consumul energetic, păstrând datele utilizatorilor la sursă.

Medical

- **Diagnostic, prognoză și imagistică:** FL este folosit în spitale și clinici pentru detectarea celulelor canceroase din imagini RMN, CT sau radiografii, fără transferul imaginilor către un server central.
- **Confidențialitate menținută la sursă:** Fiecare instituție medicală antrenează local o parte din model, partajând doar actualizările, ceea ce permite colaborarea fără a încălca regulile privind datele pacienților.

Financiar

- **Detectarea fraudelor:** Instituțiile financiare pot îmbunătăți detectarea tranzacțiilor suspecte analizând tipare comune fără a expune date sensibile despre clienți.

Servicii și experiență utilizator

- **Recomandări personalizate:** Platformele de streaming și aplicațiile mobile generează recomandări local, pe dispozitiv, fără a trimite istoricul complet al utilizatorului către server.
- **Analiză comportamentală:** FL poate analiza activitatea utilizatorilor pentru a sugera rutine sănătoase sau îmbunătățiri ale stilului de viață, păstrând confidențialitatea datelor.

Securitate și privacy

- **Supraveghere fără expunerea datelor sensibile:** Modelele de recunoaștere facială pot fi antrenate fără a transmite imagini reale, doar parametrii aferenți.
- **Analiză a sentimentelor:** FL poate analiza reacțiile utilizatorilor la evenimente sociale (like-uri, share-uri, comentarii) fără colectarea directă a acestor date de către platformă.

2.3 Atacuri de tip Data Poisoning

In acest capitol se va discuta una dintre avantajele pe care le ofera mediul federated learning atacatorilor si ce inseamna acest lucru pentru fluxul configuratiei modelului. Vom incepe cu definirea vectorilor de atac si concentrarea pe una dintre categorii, data poisoning. In continuarea lucrarii, vom analiza impactul pe care il are acest tip de atac asupra infrastructurii federate de invatare si riscurile pe care le introduce, precum si cateva propuneri de identificare si constientizare a existentei sale.

2.3.1 Definirea tipurilor de atac

In literatura de specialitate, atacurile asupra unui model de ML sau asupra unei retele neuronale sunt definite drept atacuri adversariale (adversarial attacks). Aceasta clasa de atacuri are rolul de a produce modificari in comportamentul normal al modelului intr-un mod indizerabil. Conform lucrarii [6], in functie de nivelul de scop al unui atac, putem avea:

- **Atacuri fără țintă (untargeted attacks)** Scopul este reducerea acurateții generale a modelului sau chiar destabilizarea completă a acestuia. Un exemplu pentru clasificarea imaginilor este introducerea unui zgomot care degradează calitatea setului de date. O altă metodă este modificarea etichetelor din setul de antrenare, de exemplu atribuirea etichetei *leu* unor imagini care în mod normal ar trebui etichetate drept *pisică*.

- **Atacuri țintite (targeted attacks)** Denumite și *backdoor attacks*, deoarece urmăresc modificarea comportamentului modelului doar pentru un anumit subset de date, fără a afecta vizibil acurateța globală. Continuând exemplul anterior, pentru un anumit tip de imagini ce reprezintă pisici într-o anumită poziție, se poate introduce un artefact vizual menit să păcălească modelul și să clasifice pisica drept *leu*. Astfel apare un backdoor activ doar pentru acele imagini precise.

Plecand de la aceasta categorisire, exista o multime de modalitati prin care un atacator poate submina capacitatea de predictie a modelului. Mediul federated learning introduce prin constructie o serie de intrebari la care trebuie gasit un raspuns pentru a determina prin ce moduri un adversar se poate infiltra si poate profita de anumite drepturi pentru a introduce incertitudine in antrenarea sau reglarea (fine-tuning) a modelelor.

2.3.2 Vectori de atac

Analiza amenintarilor asupra modelelor de ML, a introdus o serie de posibile vulnerabilitati asupra componentelor ce alcatuiesc fluxul de invatare. Un atacator isi poate alege zona de interes, pe baza posibilitatilor de exploatare a sistemului respectiv.

Vectorii de atac cunoscuti sunt:

- **Data Poisoning** Cand adversarul incearca sa corupa setul de date antrenare cu scopul defectarii modelului inca de la inceput.

- **Model Update Poisoning** Cand adversarul se foloseste de o vulnerabilitate ce ii permite modificarea configuratiei parametrilor trimisi catre orchestrator.

- **Evasion attack** Cand adversarul are acces la datele de testare si le poate modifica in momentul inferentei.

In functie de gradul de acces la sistemele gazda, modelul poate fi inspectat in diferite moduri:

- **Black Box** Adversarul nu are abilitatea sa inspecteze parametrii modelului inainte sau in timpul atacului.

- **Stale Box** Adversarul poate inspecta doar o versiune incipienta a modelului. Aceasta capabilitate apare si in federated learning cand adversarul are acces la rundele de antrenare ale clientului.

- **White Box** Adversarul are abilitatea de a inspecta direct parametrii modelului. Acest scenariu se bazeaza pe un grad de acces superior al adversarului asupra sistemului.

Pe baza acestor scenarii, atacatorul poate aplica o serie de tehnici pentru modificarea starii modelului. In contextul lucrarii de fata, se va discuta scenariul stale box, adversarul avand posibilitatea doar de a introduce un atac Data Poisoning in rundele de antrenare, datele corupte fiind introduse o singura data in procesul de reglare a modelului (fine-tuning).

2.3.3 Atacul Data poisoning

Acest tip de atac presupune coruperea datelor de antrenare sau de testare, prin diferite tehnici specifice, la nivelul dispozitivului clientului. In aceasta paradigma, atacul poate fi considerat la fel de bine targeted sau untargeted intrucat depinde de intentia adversarului si de potentialul risc in divulgarea punctului de exploatare de care dispune.

În practica atacurilor de tip Data Poisoning, adversarul poate manipula atât conținutul datelor, cât și etichetele acestora, efectele fiind de obicei greu de observat la nivel local. În mediul federated learning această dificultate este amplificată, deoarece orchestratorul nu are acces direct la datele brute ale clienților. Astfel, orice modificare realizată pe un dispozitiv compromis intră automat în procesul de antrenare, fiind tratată ca o contribuție legitimă. În mod particular, chiar și un număr redus de exemple otrăvite poate introduce un comportament persistent în modelul global, mai ales dacă atacul este repetat pe durata mai multor runde

In contextul solutiei propuse, se va discuta despre impactul atacului asupra unui set de imagini si tipurile de metode pentru a le altera, asa cum se poate vedea in tabelul 2.3.3.

Tip atac	**Descriere**
Gaussian noise	Introducerea unui zgomot aleator în imagini sau în vectorii de caracteristici, cu scopul degradării calității datelor și scăderii performanței modelului.
Label flip	Modificarea intenționată a etichetelor din setul de antrenare, astfel încât exemple corecte sunt asociate cu clase greșite, afectând procesul de învățare.
Backdoor injection	Inserarea unui artefact vizual sau a unui tipar specific într-un subset mic de date, astfel încât modelul să învețe un comportament anormal activat doar de acel trigger.

Tabel 2.3: Tipuri de atacuri Data Poisoning

Prin aceste tipuri de atacuri adversariale, impactul asupra modelului are loc pe o perioada determinata de timp, de obicei mai lunga, si produce variatii in predictia finala.

2.3.4 Impactul poisoning in Federated Learning

Mediul de invatare federata a introdus o serie de amenintari cibernetice preponderent la nivelul dispozitivelor clientilor, acestea fiind cele mai vulnerabile din punctul de vedere al aplicarii unui atac de otravire a datelor. Clientii sunt producatorii unui model calitativ care sa ofere predictii legitime in diferite scenarii.

Cand se discuta despre alterarea etichetelor (Label Flip Attack) atunci la o prima vedere, utilizatorul nu si-ar da seama decat in urma unei inspectii amanuntite. Pentru acest tip de atac, exista tehnici de verificare si filtrare ce pot determina daca un tip de informatie este catalogata corect inainte de antrenare.

Efectele unui data poisoning pot persista chiar și după eliminarea datelor corupte, deoarece modelul învață un tipar greșit care nu dispare imediat fără o reantrenare completă. Acest lucru este relevant mai ales pentru atacurile backdoor, care rămân inactive până la apariția unui trigger vizual, fără a afecta acuratețea generală. Din acest considerent, atacurile tintite (targeted attacks) sunt deosebit de periculoase in contextul unui mediu de invatare federata pentru ca datele corupte se ascund in interiorul configuratiilor particulare ale clientilor. Aceste configuratii sunt

transmise mai departe la agregator care aplicand functia sa de agregare, amplifica negativ starea modelului.

Având în vedere aceste particularități, devine esențială analizarea metodelor de apărare și a mecanismelor prin care pot fi detectate contribuțiile malițioase. Provocarile in acest domeniu conduc la o serie tot mai mare de utilitare sau platforme ce permit detectarea facila a acestor atacuri si imbunatatirea evenimentelor cu un potential risc in organizatii.

Mediul federated learning introduce un risc suplimentar: un adversar care controlează un număr mic de clienți poate influența disproporționat modelul global dacă este integrat într-un moment critic al antrenării. În absența unor mecanisme robuste de apărare, actualizările malițioase sunt tratate ca fiind legitime, iar agregatorul nu are nicio modalitate directă de a le verifica.

Un alt efect important al acestui tip de atac este degradarea treptată a performanței modelului. În scenariile untargeted, scăderea acurateții globale poate trece neobservată în primele runde de antrenare, dar devine evidentă odată ce modelul converge către o reprezentare eronată a datelor. În scenariile targeted, atacul poate compromite decizii critice doar într-un subset de cazuri, ceea ce face detectarea mult mai dificilă și impactul mult mai nociv, mai ales în aplicații sensibile cum ar fi securitatea, domeniul medical sau sistemele autonome.

2.4 Alte Notiuni

In vederea elaborarii solutiei propuse in aceasta lucrare, se vor aminti celelalte concepte care stau la baza implementarii propriu-zise. In acest capitol se vor detalia succint mecanismele ce stau la baza platformei propuse, modul de utilizare si scopul alegerii lor.

2.4.1 Docker

Docker reprezintă un set de servicii software de tip platformă ce utilizează virtualizarea la nivel de sistem de operare pentru a crea entități independente, numite containere. Aceste containere sunt create specific pentru a întreprinde anumite acțiuni și oferă un mediu izolat de execuție.

Un container este o instanta software ce vine impachetata cu programul aplicatiei si toate bibliotecile necesare dezvoltarii ei. O imagine este vizualizata drept un sablon de instructiuni pentru crearea unui container cu un anumit tip de biblioteci necesare dezvoltarii unei aplicatii. De obicei, imaginile sunt construite pe baza unui fisier denumit Dockerfile care propune o serie de comenzi pentru crearea unui mediu personalizat.

Toate containerle Docker ruleaza prin intermediul Docker Engine, un serviciu ce ruleaza la nivelul sistemului de operare si ofera suport cross-platform (Linux, Windows, macOS).

Containerizarea diferă de virtualizarea tradițională prin faptul că containerele partajează același kernel al sistemului de operare gazdă, în timp ce mașinile virtuale (VM) necesită fiecare un sistem de operare complet. Această abordare face containerele Docker mult mai ușoare (de ordinul MB vs GB) și mai rapide la pornire (secunde vs minute) comparativ cu VM-urile.

Arhitectura platformei docker este asemanatoare modelului client-server, fiind compusa din urmatoarele componente:

- **Dockerd** un proces daemon, identificabil drept server, ce gestioneaza tot fluxul de servicii, de la imagini si containere pana la volume si retele.
- **Aplication Binary Interfaces (API)** o suita de interfete de comunicare si control al serverului

- **Comanda docker** o interfata in linie de comanda, docker

În contextul platformei dezvoltate, Docker ofera un mediu de dezvoltare automat in care se regaseste gazduita platforma publica de simulare. Toate informatiile despre fiecare simulare sunt stocate la nivelul unei baze de date PostgreSQL, in timp ce aplicatia web este publica printr-un container frontend, iar in spate regasim un container backend pentru transmiterea de comenzi serverului.

Comunicatia dintre containere are loc in aceeasi retea locala docker, iar informatiile despre fiecare simulare a clientului persista in acelasi volum partajat.

Pe langa Dockerfile, serviciul Docker Engine ofera si posibilitatea crearii unei configuratii prestabilite pentru definirea de topologii de retea de containere prin Docker Compose. Pe baza fisierelor Dockerfile in care sunt definite sabloanele imaginilor si a unui fisier de configurare YAML a topologiei relatiilor dintre containere, serviciul Docker Compose permite implementarea unei infrastructuri intregi prin rularea si stergerea sa dintr-o serie de comenzi.

2.4.2 Python

Python este unul dintre limbajele de programare cele mai des utilizate in contextul global actual. Python este un limbaj interpretat, de nivel inalt, ce vine cu o suita de biblioteci si functii utile in mai toate domeniile.

Intreaga implementarea operatiunilor backend din proiectul propriu-zis a fost redactata folosind limbajul python, fiind nu doar util pentru a programa aplicatii software, dar intalnit in Machine Learning, pentru a antrena retele neuronale si modele de invatare automata.

2.4.2.1 Biblioteci Python utilizate

Python a devenit de facto standardul in domeniul ML si Deep Learning datorita ecosistemului sau bogat de biblioteci specializate. In contextul inteligentei artificiale, cele mai intalnite biblioteci utilizate in platforma lucrarii sunt:

- **TensorFlow**: una dintre cele mai populare biblioteci open-source pentru Machine Learning, dezvoltată de Google Brain Team. TensorFlow oferă o platformă completă pentru construirea și antrenarea modelelor de Deep Learning, cu suport nativ pentru GPU, ceea ce accelerează semnificativ procesul de antrenare. GPU sunt placi grafice dedicate procesarii unui singur set de instructiuni simultan pe multiple seturi de date (SIMD), avand integrate mai multe coruri de procesare decat un procesor normal.

- **PyTorch**: un alt framework major de Deep Learning, dezvoltat de Meta AI Research, cunoscut pentru flexibilitatea sa și abordarea dinamică a grafurilor computaționale

- **scikit-learn**: biblioteca fundamentală pentru Machine Learning tradițional în Python, oferind implementări eficiente ale algoritmilor clasici precum Random Forest, Support Vector Machines, Logistic Regression și k-Nearest Neighbors. În cadrul platformei, scikit-learn joacă un rol crucial prin calculul metricilor de evaluare prin funcții specializate precum confusion_matrix, classification_report, accuracy_score, f1_score.

- **NumPy**: constituie fundația numerică a întregului ecosistem Python pentru calcul științific. NumPy oferă suport pentru array-uri multidimensionale și o colecție vastă de funcții matematice pentru operații vectorizate.

2.4.2.2 Utilizarea TensorFlow in cadrul lucrarii

In implementarea lucrarii, TensorFlow a fost folosit in urmatoarele scopuri:

- Definirea arhitecturilor de rețele neuronale prin API-ul *Keras*, care oferă o interfață de nivel înalt pentru construirea straturilor (*layers*) și modelelor
- Antrenarea modelelor pe datele locale ale fiecărui client FL prin utilizarea de optimizori, precum *Adam*. Un optimizor este o metoda matematica de reducere a ratei de eroare si de imbunatatire a acuratetii parametrilor modelului de invatare automata.
- Extragerea și setarea ponderilor (*weights*) modelului pentru procesul de agregare, operatiune esentiala in FL
- Evaluarea performanței modelului prin metrici precum *accuracy*, *loss*, *precision* și *recall*.

2.4.2.3 Metrici de Evaluare in Machine Learning

Acuratețea (*accuracy*) este definită ca raportul dintre numărul de predicții corecte (atât *true positives*, cât și *true negatives*) și numărul total de predicții:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}} = \frac{TP+TN}{TP+TN+FP+FN} \tag{2.1}$$

Recall, sau true positive rate (TPR) este metrica ce determina procentul de date care a fost clasificat corect. Aceasta valoare include la numitor atat numarul de elemente a caror predictie a fost identificata corect drept adevarata si numarul elementelor prezise corect drept incorecte.

$$\text{TPR} = \frac{TP}{TP+FN} \tag{2.2}$$

Precizia exprimă proporția predicțiilor pozitive ale modelului care sunt corecte:

$$\text{Precision} = \frac{TP}{TP+FP} \tag{2.3}$$

Anaconda facilitează instalarea și actualizarea rapidă a bibliotecilor necesare, asigurând compatibilitatea între diferite versiuni și reducând riscul apariției conflictelor de dependințe. La nivelul lucrarii, au fost create două medii distincte în Anaconda, fiecare configurat cu bibliotecile relevante pentru proiect, TensorFlow si PyTorch, utilizate in procesul de simulare.

Dincolo de capacitățile sale în Machine Learning, Python excelează și în dezvoltarea de aplicații web și servicii backend. În cadrul platformei, Python îndeplinește multiple roluri care vor fi amintite pe tot parcursul capitolului 3 de implementare.

2.4.3 Rest API

REST (Representational State Transfer) API reprezintă o arhitectură software pentru comunicarea între componente distribuite prin protocolul HTTP. Un API (Application Programming Interface) reprezintă o interfață definita special prin care diferite componente software comunică între ele.

În platforma dezvoltată, REST API asigură comunicația între frontend (React), backend (FastAPI) și orchestrator (Flask), implementând un sistem complet de management al simulărilor federate.

2.4.3.1 Arhitectura REST in Platforma

Platforma utilizează două servere REST API distincte, fiecare cu responsabilități specifice:

Backend API (FastAPI) rulează în containerul Docker local și servește ca intermediar între interfața utilizator și serverul de simulare. FastAPI este un framework Python modern, de înaltă performanță, bazat pe Pydantic pentru validarea automată a datelor și pe specificația OpenAPI pentru documentare automată. Alegerea FastAPI față de Flask pentru backend este justificată de performanța superioară în scenarii cu cereri concurente, suportul nativ pentru operații asincrone.

Orchestrator API (Flask) rulează pe serverul ATM și gestionează execuția efectivă a simulărilor. Flask este un framework web minimalist, ideal pentru servicii interne care nu necesită complexitatea FastAPI. Orchestratorul primește comenzi de la backend, alocă resurse GPU, lansează procese de simulare, și returnează rezultatele.

Un aspect fundamental al arhitecturii este comunicarea asincronă între componente. Simulările FL pot dura de la minute la ore, făcând imposibilă așteptarea sincronă a rezultatelor. Platforma implementează pattern-ul Job Queue cu următoarele caracteristici:

- Inițierea simulării se realizează prin cererea POST către backend la endpoint-ul dedicat, stochează configurația în baza de date PostgreSQL, și transmite cererea către orchestrator.
- Monitorizarea progresului se realizează prin două mecanisme complementare: WebSocket pentru actualizări în timp real și polling HTTP pentru backwards compatibility.

Backend-ul expune multiple categorii de endpoint-uri, fiecare responsabil de o funcționalitate specifică:

- Autentificare și Autorizare se realizează prin endpoint-uri dedicate pentru înregistrare utilizatori, autentificare cu username și parolă, și generare JWT (JSON Web Token) pentru sesiuni. Token-ul JWT este inclus în header-ul Authorization pentru toate cererile ulterioare, sub forma Bearer token. Backend-ul validează token-ul pentru fiecare cerere protejată, verificând semnătura digitală, expirarea, și existența utilizatorului în baza de date.
- Management Proiecte și Fișiere permite utilizatorilor să organizeze simulările în proiecte ierarhice. Fiecare proiect poate conține multiple fișiere reprezentând template-uri Python pentru modele diferite.
- Execuția Simulărilor este orchestrată prin endpoint-ul principal care primește codul template Python și configurația simulării. Backend-ul efectuează validări de securitate asupra codului încărcat pentru a preveni atacurile de tip code injection, verifică existența funcțiilor obligatorii în template și transmite cererea către orchestrator împreună cu token-ul de autentificare.
- Colectarea Rezultatelor se realizează prin endpoint-uri care interogează orchestratorul pentru obținerea rezultatelor finale. După ce orchestratorul semnalează completarea simulării, backend-ul solicită fișierele JSON cu metrici, fișierul text cu rezumatul, și informații despre atacul aplicat.
- WebSocket pentru Monitorizare Real-Time implementează un endpoint special care acceptă conexiuni WebSocket. După stabilirea conexiunii, backend-ul intră într-un loop asincron în care interoghează orchestratorul la fiecare 2 secunde pentru status-ul actualizat al simulării. Informațiile sunt trimise imediat către frontend prin WebSocket, oferind o experiență interactivă similară unui terminal live.

Capitolul 3:

Proiectare, Implementare si Testare

Lucrarea se concentreaza pe dezvoltarea unei platforme de simulare a atacurilor de tip Data Poisoning intr-un mediu de invatare automata federata. Scopul ei vine chiar din problema identificarii acestui tip de atac si imbunatatirea procesului de alertare in securitatea cibernetica. Platforma sprijina analistii de securitate si cercetatorii de analiza de date in vederea constientizarii unei posibile vulnerabilitati la nivelul unei infrastructuri publice centralizate.

3.1 Arhitectura Platformei

In procesul de dezvoltare al platformei, au fost analizate platforme de programare in timp real, precum Google Colab sau Jupyter Notebook, ce permit utilizatorilor sa isi incarce si sa execute remote codul de antrenare al retelelor neuronale intr-o infrastructura specializata. Inspirandu-se din aceste platforme si observand existenta unor utilitare de analiza a malware-ului si infectiilor, s-a identificat nevoia unui framework care nu doar sa simuleze o retea de invatare automata federata, ci si sa testeze atacuri Data Poisoning oferind cercetatorilor si analistilor de securitate posibilitatea de a evalua vulnerabilitatile propriilor retele neuronale.

Platforma este structurată pe trei niveluri principale:

Nivelul Client (Frontend)

- Interfață React pentru autentificare, management proiecte, și vizualizare rezultate
- Comunicare prin REST API și WebSocket pentru monitorizare in timp-real.

Nivelul Backend (Containerizat Docker)

- API FastAPI pentru gestionarea cererilor utilizatorilor
- Baza de date PostgreSQL pentru persistența configurațiilor și rezultatelor
- Validare securitate cod și transmitere asincronă către orchestrator

Nivelul Execuție (Server ATM)

- Orchestrator Flask pentru managementul simulărilor
- Alocare dinamică GPU prin GPUManager (3 GPU-uri disponibile)
- Medii Python izolate: fl_tensorflow și fl_pytorch

Fluxul platformei este urmatorul:

- Utilizatorul se autentifica in platforma si isi creeaza un proiect si un fisier de test aferent, in care incarca un sablon specific pentru antrenare. Sablonul este compus dintr-o serie de functii predefinite care vor trebui completate cu secvente de cod specifice crearii retelei neuronale, antrenarii si evaluarii ei. Printre alte functii, trebuie sa amintim si de cele de setare a ponderilor pentru procesul de agregare la nivelul infrastructurii federate, de descarcare de date local de la o sursa specifica si de salvare a configuratiei retelei antrenate.

- Odata ce se initiaza simularea la apasarea butonului Run, sablonul python este transmis asincron de la backend-ul aplicatiei catre un server remote. Comunicarea asincronă permite utilizatorului să continue lucrul în platformă în timp ce simularea se execută pe server. Serverul remote este denumit Server ATM, fiind localizat in infrastructura Academiei Tehnice Militare "Ferdinand I". Pe serverul remote, cererea este preluata de un proces orchestrator care odata primita, instantiaza un nou proces care executa o serie de actiuni.

- Odata initiata o cerere de simulare, se va crea in directorul utilizatorului logat in platforma, un director specific denumit dupa identificatorul taskului instantiat de utilizator. In acel director, se afla toate tipurile de fisiere care vor fi gestionate pe tot parcursul operatiunile de simulare, de la setul de date de antrenare, pana la fisierele specifice tipului de atac Data Poisoning utilizat si rezultatele simularilor in reteaua federata.

- Prima etapa este cea in care se identifica tipul de biblioteci utilizate pentru procesarea retelei neuronale si se trece in mediul python corespunzator. Platforma suporta antrenarea folosind biblioteci TensorFlow si PyTorch, avand create cate un mediu specific pentru antrenarea tipului de retea respectiv. Scriptul de rulare a retelei neuronale se numeste train_model.py si este un script agnostic, construit pentru a testa functiile din sablon.

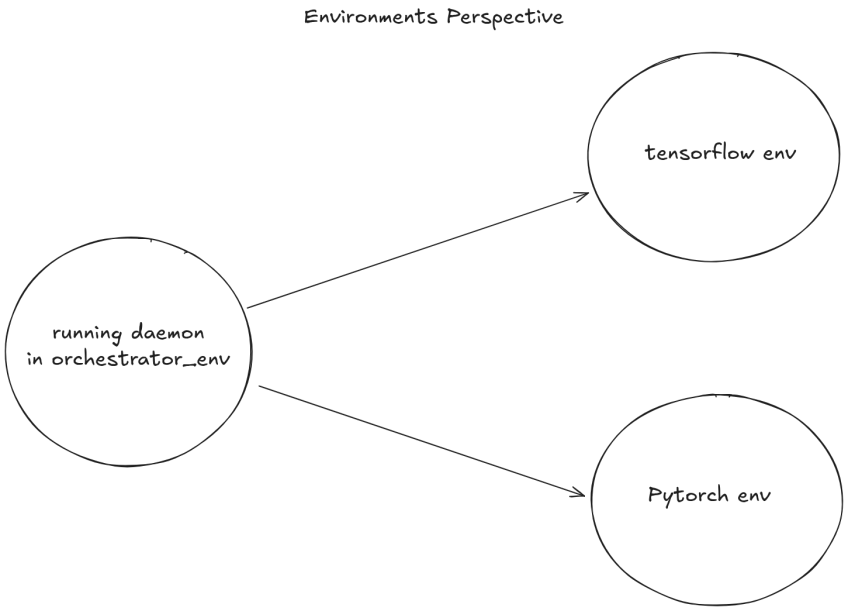

Figura 3.1: Medii de python platforma

- Imaginea de mai sus reprezinta mediile de python disponibile pentru a rula retelele neuronale.

- La fiecare pas, in cazul unei erori generate de orice fel de actiune, procesul ce gestioneaza operatiunile de simulare se opreste si transmite eroarea ca rezultat procesului orchestrator care semnalizeaza backend-ul aplicatiei.

- Dupa prima antrenare, se salveaza parametrii aferenti matricii de confuzie (acuratete, precizie, scor F1, recall) pentru a compara rezultatul simularii cu configuratia initiala a retelei si detectarea posibilelor anomalii. Acesta este principalul pas in procesul de analiza a problemei propuse.

- Daca programul este corect structurat, procesul curent incarca functiile scriptului transmis si se foloseste de ele pentru a simula atacul Data Poisoning. Mai intai se apeleaza functia de descarcare locala a datelor de antrenare si testare. Aceste date vor fi folosite pentru a simula procesul de otravire pe baza configuratiei simularii setate de utilizator. Datele pot fi descarcate din orice sursa publica, permitand o flexibilitate in tipul de platforma si de date pentru testare.

- Argumentele cu ajutorul carora se ruleaza scriptul de otravire a setului de date sunt:

  - **poison_operation**: tipul atacului de otrăvire a datelor. Platforma suportă 7 tipuri de atacuri:

    * *Label Flip* — inversarea etichetelor unui procent de imagini către o clasă țintă
    * *Backdoor BadNets* — inserarea unui trigger vizual mic (cruce, pătrat) pe imagini
    * *Backdoor Blended* — combinarea imaginii originale cu un pattern cheie folosind un raport configurabil
    * *Backdoor Sinusoidal* — adăugarea unui semnal sinusoidal cu frecvență și amplitudine configurabile
    * *Backdoor Trojan* — inserarea unui watermark în imagini
    * *Backdoor Semantic* — modificarea caracteristicilor naturale ale imaginii (luminozitate, tonalitate RGB)
    * *Edge-Case Backdoor* — aplicarea de transformări asupra imaginilor (rotație, reducere culori, solarizare)

  - **poison_intensity**: intensitatea atacului folosit doar in cazul atacurilor backdoor (ex: 0.1 pentru zgomot Gaussian, dimensiune trigger pentru backdoor)

  - **poison_percentage**: procentajul de date afectate per client (ex: 0.2 pentru 20% date compromise la nivelul unui client malitios)

- Amintim ca mediile de python includ si biblioteci de seturi de date specifice precum TensorFlow Datasets in care se regasesc seturi de date predefinite in cazul in care utilizatorul doreste sa foloseasca un set de date predefinit la nivel de biblioteci.

- Procesul de otravire (poisoning) presupune rularea unui script care pe baza parametrilor simularii, aplica o serie de operatii asupra seturilor de date. Tipurile de actiuni de otravire au fost definite in capitolul 2 (sectiunea 2.3.3 - Atacul Data poisoning). Utilizatorul poate alege procentajul din etichete care sa fie infectate, iar in cazul atacurilor backdoor, se vor seta parametrii specifici fiecărui tip de trigger.

- In pasii urmatori, se va realiza simularea propriu-zisa cu ajutorul unui script agnostic denumit fl_simulator, care va primi drept argumente urmatoarele:

  - N numarul de clienti/dispozitive aferente retelei federate
  - M numarul clienti compromisi din totalul de mai sus
  - R numarul total de runde de antrenare
  - ROUNDS numarul de runde in care dispozitivele compromise vor antrena cu datele corupte
  - strategia de propagare a dispozitivelor compromise:

    * *first*: dispozitivele compromise vor rula primele ROUNDS runde cu datele otravite, iar restul de R-ROUNDS runde vor folosi datele originale
    * *last*: dispozitivele compromise vor rula ultimele ROUNDS runde cu datele otravite, iar restul de R-ROUNDS runde vor folosi datele originale
    * *alternate*: dispozitivele compromise vor rula cate o runda folosind datele otravite si cate una cu datele originale
  - metoda de agregare robustă, selectată de utilizator:
    * *FedAvg* — media ponderată a actualizărilor clienților, proporțională cu dimensiunea setului de date
    * *Krum* — selectează actualizarea clientului cu distanța totală minimă față de celelalte, izolând contribuțiile malițioase
    * *Trimmed Mean* — elimină primele și ultimele 20% valori extreme ale ponderilor, apoi calculează media restului
    * *Median* — înlocuiește media cu mediana per parametru, fiind rezistentă la aproximativ 20% clienți malițioși
    * *Trimmed Mean + Krum* — abordare hibridă care combină filtrarea Trimmed Mean cu selecția Krum
    * *FoolsGold* [8] — calculează similaritatea cosinus între actualizările clienților de-a lungul procesului de antrenare și penalizează (reduce ponderile de agregare) pentru contribuțiile cu o similaritate ridicată, reușind să neutralizeze eficient atacurile coordonate (de tip sybil)
    * *Norm Clipping* — limitează magnitudinea (norma L2) a vectorului de actualizare trimis de un client la un prag maxim prestabilit, oprind astfel actualizările malițioase cu valori extreme din a domina procesul de agregare
    * *Random* — selectează aleator ponderile unui client, utilizat ca bază de comparație

- Simularea propriu-zisă execută **patru scenarii distincte**, fiecare pornind de la aceeași configurație inițială a modelului pentru a asigura comparabilitatea rezultatelor:

  1. **Simulare cu date curate (Clean)** — toate dispozitivele folosesc date nemodificate, iar agregarea se face cu FedAvg. Acest scenariu stabilește performanța de referință (baseline) în condiții ideale, fără atac.
  2. **Simulare cu date curate și apărare (Clean + Defense)** — datele rămân nemodificate, dar se aplică metoda de agregare robustă selectată de utilizator. Acest scenariu măsoară costul computațional (overhead-ul) introdus de mecanismul de apărare în absența unui atac.
  3. **Simulare cu date otrăvite (Poisoned)** — M din N clienți folosesc date otrăvite, iar agregarea folosește FedAvg (fără apărare). Acest scenariu cuantifică impactul atacului data poisoning asupra modelului global.
  4. **Simulare cu date otrăvite și apărare (Poisoned + Defense)** — același scenariu de atac, dar cu agregarea robustă activată. Comparând rezultatele cu scenariul anterior, se evaluează eficacitatea strategiei de apărare selectate.

- In urma celor patru simulari, se vor salva informatiile cu privire la fiecare configuratie simulata, ce va fi utilizata in procesul de analiza.

  Analiza rezultatelor presupune compararea următoarelor metrici între cele patru scenarii:

- **Acuratețea** — valorile de acurateț pentru fiecare scenariu (Inițial, Clean, Clean+Defense, Poisoned, Poisoned+Defense)
- **Degradarea acurateții** — diferențele între scenarii pentru cuantificarea impactului atacului și a eficacității apărării:
  * Degradare absolută: $\Delta_{acc}^{abs} = Acc_{clean} - Acc_{poisoned}$
  * Variații față de modelul inițial: diferența dintre modelul final din fiecare scenariu și acuratețea pre-antrenată de referință (ex. $Acc_{clean} - Acc_{init}$, $Acc_{poisoned} - Acc_{init}$)
- **Metrici de clasificare**: Precision, recall și F1-score (medie ponderată) pentru fiecare scenariu

Rezultatele sunt salvate în trei fișiere: `analysis.json` (metrici structurate), `summary.txt` (rezumat text asupra simularilor) și `attack_info.json` (detalii despre atac).

- Utilizatorul va putea vizualiza fiecare pas de procesare amintit pana acum si rezultatul analizei simularii.
- Pentru a facilita munca analistului, platforma include posibilitatea de export a raportului simularii în format PDF și CSV, dar si compararea de simulari ale aceluiasi proiect.

3.2 Implementare Platformei

Platforma este compusă din două servere, unul dintre ele fiind remote și dispunând de trei plăci grafice NVIDIA A40 (GPU) optimizate pentru medii de lucru cu Inteligență Artificială și Deep Learning. Plăcile dispun de 46 GB memorie VRAM (video RAM) GDDR6 pentru a suporta lucrul intensiv cu date. A40 este o placă profesională destinată centrelor de date, furnizând accelerare hardware pentru TensorFlow și PyTorch, precum și pentru operații matematice complexe necesare în procesele de simulare și învățare federată.

Serverul care rulează funcționalitățile backend și de persistență se află în rețeaua locală proprie. Acesta este responsabil pentru gestionarea interacțiunii utilizatorilor, stocarea configurațiilor și transmiterea cererilor către serverul remote. El reprezintă punctul de intrare al platformei pentru toți utilizatorii și menține logica de securitate, autentificare și gestionare a sesiunilor.

Pentru scalabilitate și flexibilitate, s-au utilizat containere Docker la nivelul serverului local pentru a stoca următoarele componente:

- **Containerul frontend** - include toate dependințele necesare pentru rularea unei aplicații React folosind un server NodeJs. Acesta oferă o interfață modernă și receptivă, permițând utilizatorilor accesarea tuturor funcțiilor platformei, precum încărcarea de șabloane, vizualizarea rezultatelor și compararea simulărilor.
- **Containerul bazei de date PostgreSQL** - stochează toate fișierele cu rezultatele simulărilor executate pe server, specifice fiecărui utilizator, precum și parolele hash ale utilizatorilor. Structura bazei de date este optimizată pentru volum mare de date și interogări rapide asupra istoricului de simulări.
- **Containerul backend** - procesul responsabil de primirea cererilor API. Backend-ul recepționează comenzi de la utilizatori sub forma unei cereri POST către endpoint-ul `/run`. După validare, cererea este retransmisă asincron către serverul remote. Backend-ul implementează

validări de securitate asupra șabloanelor, logarea fiecărei cereri și mecanisme de prevenire a supraîncărcării serverului remote.

La nivelul serverului remote, platforma include următoarele componente:

- **Procesul orchestrator** - rulează scriptul `orchestrator_gpu.py`, care creează un proces nou pentru fiecare conexiune inițiată. Pentru fiecare simulare se alocă un GPU, asigurând un flux de procesare ridicat fără a afecta timpul de execuție al simulărilor deja active. Orchestratorul gestionează coada de task-uri, starea proceselor și comunicarea bidirecțională cu serverul local. Pipeline-ul de simulare parcurge 14 etape secvențiale, de la verificarea structurii șablonului, alocarea resurselor și descărcarea datelor, până la execuția celor patru scenarii de simulare și generarea rezultatelor finale.
- **Managerul de GPU** - implementat în scriptul `gpu_manager.py`, monitorizează în timp real starea GPU-urilor (nivel de utilizare, memorie liberă, temperatură) și distribuie simulările în mod echilibrat pentru a preveni supraîncărcarea unuia dintre dispozitive. Fiecare simulare solicită un GPU dedicat cu un timeout de 10 minute; dacă niciunul nu este disponibil în acest interval, simularea trece automat pe CPU. GPU-ul este **garantat eliberat** la finalizarea, anularea sau eșecul simulării.
- **Procesul de simulare a atacului** - identifică tipul bibliotecilor Python folosite în șablonul primit și comută automat în mediul Anaconda corespunzător (TensorFlow sau PyTorch). Mediile Python sunt complet izolate, având propriile dependințe pentru a asigura reproducibilitatea rezultatelor. Otrăvirea datelor este realizată prin scriptul `poison_data.py`, care suportă 7 tipuri de atacuri cu parametri configurabili per atac. Simularea propriu-zisă rulează prin `fd_simulator.py`, care utilizează șablonul de cod stocat în `template_code.py`. În timpul execuției, fiecare etapă este jurnalizată, iar eventualele erori sunt transmise backend-ului în timp real.
- **Fluxul de anulare** - la fiecare etapă din pipeline, orchestratorul verifică dacă utilizatorul a solicitat anularea. În caz afirmativ: arborele de procese este terminat (`kill_process_tree()`), GPU-ul alocat este eliberat, directorul de lucru al simulării este șters, iar statusul este marcat ca *anulat*.

Etapele procesului de simulare

Atunci când un utilizator inițiază o simulare de învățare federată, orchestratorul de pe serverul GPU declanșează un pipeline secvențial compus din 14 etape distincte:

1. **Pornirea task-ului de simulare (Start Simulation Task)**: Orchestratorul creează o nouă intrare pentru task și începe execuția asincronă a pipeline-ului.
2. **Verificarea structurii șablonului (Verify Template Structure)**: Înainte de a consuma resurse computaționale (cum ar fi GPU), se efectuează o analiză statică suplimentară a scriptului Python de antrenare furnizat de utilizator, folosind modulul intern `verify_template.py`.
3. **Crearea directorului de lucru (Create User Workspace)**: Este creat un director izolat pe disc, specific task-ului curent, pe baza ID-ului utilizatorului și a identificatorului unic al simulării.

4. **Verificarea corectitudinii modelului (Verify Model Correctness)**: Utilizând codul șablon primit, orchestratorul instanțiază modelul definit de utilizator și rulează o scurtă încercare de antrenare pentru a valida funcționalitatea acestuia local, înainte de modelul federativ.

5. **Descărcarea modelului din repozitoriu (Download Model)**: Se apelează logica de definire a modelului din șablon; dacă modelul este pre-antrenat și găzduit extern (ex. HuggingFace), acesta este descărcat și salvat de pe server în formatul specific framework-ului detectat.

6. **Descărcarea setului de date (Download Dataset)**: Orchestratorul apelează funcția de descărcare și pregătire a datelor (testare și antrenare) propusă în codul utilizatorului.

7. **Alocarea unității de procesare grafică (Allocate GPU)**: Managerul de GPU caută o placă video NVIDIA disponibilă în sistem, procesând elementele dintr-o coadă de așteptare. Dacă timpul alocat expiră, se utilizează unitatea CPU.

8. **Activarea mediului Python (Activate Python Environment)**: Prin analiza importurilor din șablon, orchestratorul detectează tipul aplicatiei (TensorFlow sau PyTorch) și activează mediul *Conda* preconfigurat corespunzător.

9. **Alterarea datelor de antrenare (Poison Dataset)**: Folosind scriptul `poison_data.py`, orchestratorul creează o copie malformată a datelor destinate antrenării, aplicând exact tipul de atac, procentajul și intensitatea alese din interfață.

10. **Simularea în mediul curat (Clean Simulation)**: Modelul global intră în faza de simulare federată (folosind `fd_simulator.py`). Procesul asincron este inițiat exclusiv pe setul de date nealterate (curat), partajat pe N clienți legitimi, folosind algoritmul standard de agregare, *FedAvg*. Această etapă înregistrează performanța normală (de referință).

11. **Simularea curată cu mecanism de apărare (Clean + Defense)**: Continuându-se procesele pentru testare, *FedAvg* este înlocuit acum cu tehnica robustă de agregare aleasă de utilizator (ex. FoolsGold, Norm Clipping), tot pe setul curat de date.

12. **Simularea cu atac fără apărare (Poisoned Simulation)**: Este integrat setul alterat (din etapa 9). M dispozitive capătă rol de clienți malițioși și raportează parametrii compromiși către orchestrator, agregarea făcându-se prin *FedAvg*. Aceasta va măsura impactul loviturii asupra performanței totale.

13. **Simularea sub atac și cu protecție activă (Poisoned + Defense)**: Aceeași simulare cu M clienți atacatori este reluată folosind de data aceasta metoda de agregare robustă ce trebuie să filtreze ponderile malițioase.

14. **Generarea rezultatelor finale (Generate Final Results)**: Orchestratorul cumulează rezultatele din toate simulările (scenariul normal, overheadul apărării, capacitatea de distrugere a atacului și reușita apărării) și generează rapoarte structurate, oferite utilizatorului direct pe ecran.

Arhitectura propusă oferă următoarele beneficii:

- **Flexibilitate**: Sistemul de șabloane permite testarea oricărui model TensorFlow sau PyTorch fără modificări ale infrastructurii. Izolarea mediilor Python elimină conflictele dintre versiuni diferite ale bibliotecilor.

- **Scalabilitate**: Alocarea dinamică a GPU-urilor permite rularea simultană a trei simulări, maximizând utilizarea resurselor disponibile. Structura modulară permite extinderea ulterioară cu mai multe GPU-uri sau chiar cu mai multe servere.
- **Reproducibilitate**: Toate configurațiile și rezultatele sunt stocate în PostgreSQL, permițând replicarea exactă a experimentelor. Mediile conda imutabile garantează consistența dintre execuții.
- **Transparență**: Monitorizarea în timp real prin WebSocket oferă vizibilitate completă asupra procesului de execuție, facilitând depanarea eventualelor probleme.
- **Comparabilitate**: Funcționalitatea de comparare a simulărilor permite evaluarea impactului diferitelor tipuri de atacuri asupra aceluiași model, datele fiind centralizate într-o structură unificată pentru analize statistice avansate.
- **Securitate**: Prevenirea execuției de cod malițios se realizează printr-un proces strict de analiză statică a codului, încă din etapa pre-orchestrării. Atunci când utilizatorul încarcă șablonul Python, se rulează utilitarul avansat de analiză de securitate `semgrep` alături de scriptul personalizat `verify_template.py`. Aceste elemente identifică vulnerabilitățile codului propriu-zis de manipulare și transfer, descoperă injecțiile (*code injection*) și validează prezența obligatorie a structurilor critice, blocând execuțiile periculoase chiar de la nivelul backend-ului.

Imaginea 3.2 prezinta intregul flux al arhitecturii, precum si detaliile tehnice utilizate.

3.3 Cerintele Software

Cerințele software ale platformei au fost documentate conform standardului IEEE 830-1998 pentru Specificații ale Cerințelor Software (SRS). Fiecare cerință este identificată printr-un cod unic: CF-XX pentru cerințe funcționale, CNF-XX pentru cerințe nefuncționale și CS-XX pentru cerințe de securitate. Cerințele sunt clasificate după gradul de obligativitate: **OBLIGATORIU** (Faza 1), **RECOMANDAT** (Faza 2) și **OPȚIONAL** (Faza 3).

3.3.1 Cerintele functionale

Cerințele funcționale definesc comportamentul specific al platformei și sunt organizate pe module de funcționalitate. În continuare sunt prezentate principalele categorii.

3.3.1.1 Gestionarea și Autentificarea Utilizatorilor (CF-01 – CF-15)

Platforma implementează un sistem complet de management al utilizatorilor:

- Proces de înregistrare cu colectare de informații pe două niveluri: date obligatorii (nume, prenume, email, parolă) și date opționale (organizație, domeniu cercetare)
- Autentificare bazată pe email și parolă, cu parole stocate în format hash folosind algoritmul bcrypt
- Generare și gestionare de token-uri JWT (JSON Web Tokens) pentru sesiuni, cu expirare după 24 de ore de inactivitate

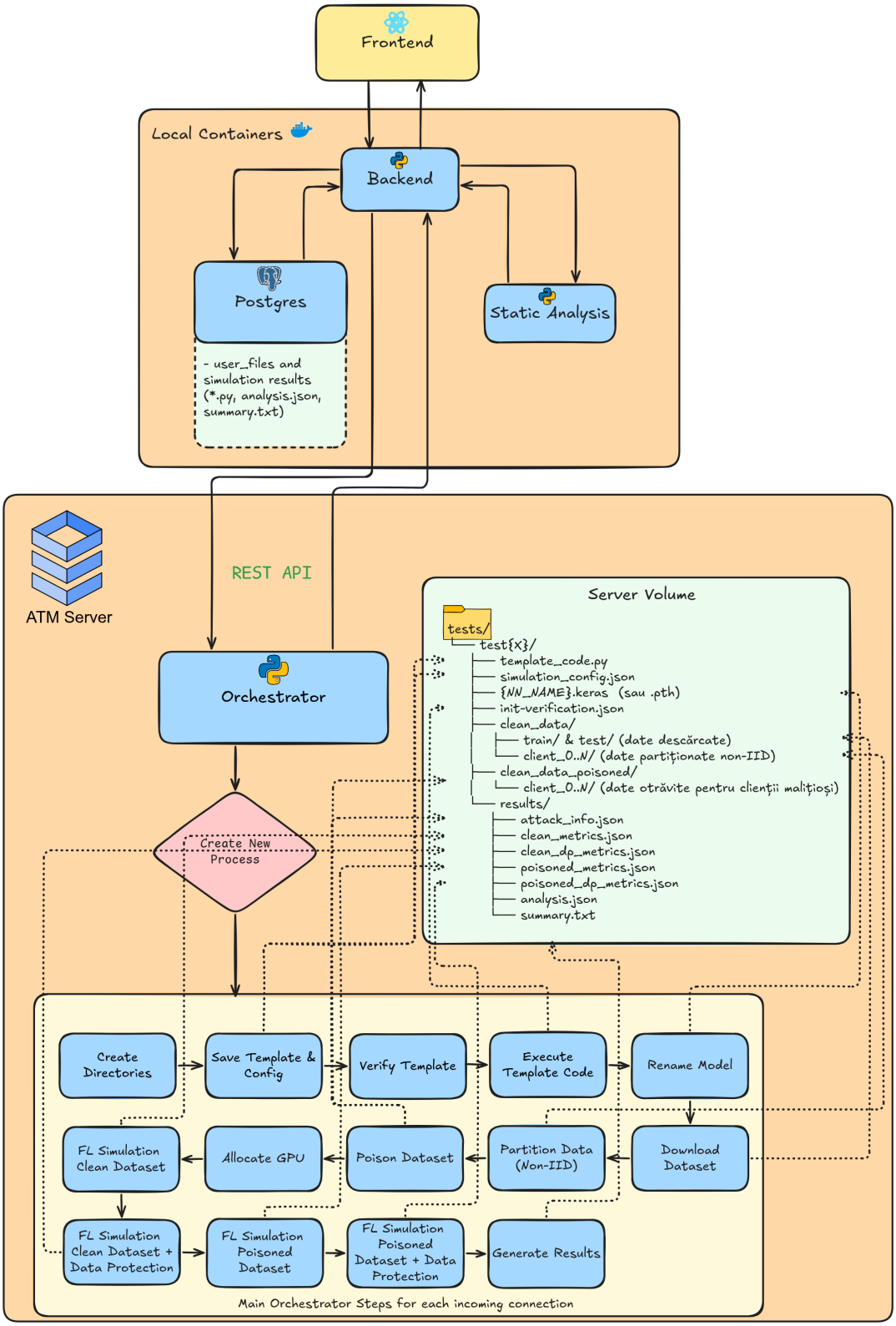

Figura 3.2: Arhitectura si fluxul platformei

- Autorizare bazată pe roluri (RBAC) cu două roluri: utilizator standard și administrator
- Validare formatul email-ului și complexitatea parolei la înregistrare (minimum 8 caractere, litere mari/mici, cifre)

- Funcționalitate de logout care invalidează token-ul JWT al utilizatorului
- Interfață în care utilizatorul își alege parametrii de simulare ai atacului și ai infrastructurii federate

3.3.1.2 Parametrii de Simulare și Vectori de Atac (CF-16 – CF-25)

Sistemul permite configurarea completă a mediului de simulare:

- Numărul total de dispozitive/clienți participanți (range: 2–100)
- Numărul de dispozitive compromise (range: 1 până la 50% din total)
- Numărul total de runde de antrenare (range: 1–500) și numărul de runde corupte per dispozitiv
- Strategia de distribuire a dispozitivelor corupte (primele, ultimele, distribuție uniformă)
- Suport pentru 7 tipuri de atacuri: Label Flip, Backdoor BadNets, Backdoor Blended, Backdoor Sinusoidal, Backdoor Trojan, Backdoor Semantic și Edge-Case Backdoor
- Configurarea intensității atacului specific fiecărui tip (deviație standard, procentaj etichete, dimensiune trigger)
- Selecția funcției de agregare: FedAvg, Krum, Trimmed Mean, Median, Trimmed Mean + Krum, Random
- Validarea valorilor parametrilor introduși cu mesaje de eroare pentru valori invalide

3.3.1.3 Interfețe de Comunicare (CF-26 – CF-29)

- Rețea de containere Docker interconectate pentru comunicarea frontend-backend
- REST API pentru comunicarea dintre serverul central și serverul de simulare
- Monitorizarea procesului de simulare prin mesaje de logging folosind WebSockets
- Mecanism de load-balancing prin crearea de task-uri asincrone pentru simulări multiple

3.3.1.4 Integrarea Tehnologiilor (CF-30 – CF-33)

- Editor de text performant (Monaco Editor) pentru editarea scripturilor Python
- Biblioteci de vizualizare a informației (Bar Chart) pentru analiza rezultatelor
- Suport pentru simularea cu modele TensorFlow și PyTorch
- Suport pentru încărcare de modele preantrenate din repository-uri publice (HuggingFace)

3.3.1.5 Interpretarea Rezultatelor (CF-34 – CF-38)

- Afișarea rezultatelor simulării cu metrici detaliate (acuratețe, precizie, recall, F1-score) per scenariu
- Compararea side-by-side a două simulări, inclusiv configurația FL, parametrii atacului și toate metricile
- Vizualizare grafică interactivă cu bar charts grupate pentru mai multe simulări
- Export rezultate individuale în format PDF (configurație, metrici, matrice de confuzie, rezumat)
- Export multiplu în format CSV cu toate metricile per scenariu în format tabular

3.3.1.6 Gestionarea Proiectelor și Fișierelor (CF-39 – CF-48)

- Creare și organizare ierarhică a proiectelor de simulare
- Creare, ștergere, redenumire și reatribuire a fișierelor de simulare între proiecte
- Stocare proiecte și fișiere în baza de date PostgreSQL cu izolarea datelor per utilizator
- Reordonare fișiere prin drag-and-drop

3.3.1.7 Executarea și Monitorizarea Simulărilor (CF-49 – CF-56)

- Transmiterea scriptului de simulare și parametrilor către server prin REST API
- Execuție pe server dedicat cu resurse GPU alocate dinamic
- Afișarea progresului în timp real (rundă curentă, GPU alocat, status) prin WebSocket
- Anulare simulări în curs de execuție cu eliberarea automată a resurselor
- Gestionarea erorilor cu mesaje descriptive transmise utilizatorului
- Stocare rezultate finalizate în baza de date pentru consultare ulterioară

3.3.1.8 Gestionarea Modelelor (CF-57 – CF-62)

- Furnizare șablon de cod Python predefinit la crearea unui fișier de simulare, cu funcții necesare antrenării
- Încărcare modele preantrenate din HuggingFace cu descărcare automată în timpul simulării
- Editare și personalizare a funcțiilor din șablon pentru adaptarea la specificul modelului
- Validare sintaxa codului Python înainte de lansarea simulării și semnalarea erorilor
- Documentație inline și comentarii explicative în șablon pentru ghidarea utilizatorului

3.3.1.9 Procesarea și Orchestrarea pe Server (CF-63 – CF-72)

- Recepționare asincronă a cererilor de simulare prin REST API
- Creare proces dedicat pentru fiecare cerere cu toate informațiile necesare
- Descărcare model/rețea preantrenată din repository-ul specificat
- Încărcare model în memoria GPU alocat specific task-ului
- Descărcare set de date și aplicare atac data poisoning conform parametrilor
- Execuție antrenare în patru scenarii distincte: Clean, Clean+Defense, Poisoned, Poisoned+Defense
- Returnare rezultate antrenare cu metrici complete și monitorizare continuă a proceselor

3.3.2 Cerintele nefunctionale

Cerințele nefuncționale definesc atributele calității și performanței pe care sistemul trebuie să le satisfacă.

3.3.2.1 Cerințe de Performanță (CNF-01 – CNF-17)

- Interfața web va răspunde la acțiunile utilizatorului în maximum 2 secunde pentru operații standard
- API-ul backend va procesa cererile de autentificare în maximum 1 secundă
- Încărcarea dashboard-ului principal va fi completă în maximum 3 secunde
- Actualizările în timp real prin WebSocket vor avea o latență de maximum 500ms
- Sistemul va suporta minimum 50 de utilizatori concurenți fără degradare semnificativă
- Serverul de simulări va putea procesa cel puțin 5 simulări simultane pe un singur GPU
- API-ul va gestiona minimum 100 de cereri pe secundă în condiții normale
- Fiecare simulare va fi limitată la maximum 10 GB memorie GPU
- Containerizarea cu Docker va permite deploy-uri reproductibile în diferite medii

3.3.2.2 Cerințe de Securitate (CS-01 – CS-17)

- Autentificare bazată pe JWT cu expirare configurabilă, semnate cu algoritm HS256
- Parole stocate în format hash folosind algoritmul bcrypt
- Control de acces bazat pe roluri (RBAC) cu roluri user și admin
- Comunicații criptate folosind TLS 1.2 sau superior
- Izolarea datelor între utilizatori (multi-tenancy)
- Respectarea principiilor GDPR privind colectarea și procesarea datelor personale

- Scripturile de simulare încărcate vor fi executate în medii izolate (sandbox)
- Înregistrarea tuturor încercărilor de autentificare și acțiunilor administrative în jurnal de audit

3.3.2.3 Fiabilitate și Disponibilitate (CNF-18 – CNF-32)

- Disponibilitate țintă de 99% în orele de lucru (8:00–22:00)
- Implementare graceful degradation — eșecul unei componente nu va afecta întregul sistem
- Conexiunile WebSocket vor avea mecanism de reconectare automată
- Backup-uri ale bazei de date efectuate zilnic
- Erorile vor fi afișate utilizatorilor în mesaje clare, iar detaliile tehnice vor fi înregistrate în log-uri
- Simulările eșuate vor genera notificări cu detalii despre cauza erorii

3.3.2.4 Accesibilitate (CNF-33 – CNF-41)

- Interfață intuitivă ce permite utilizatorilor noi să lanseze o simulare în maximum 10 minute
- Feedback vizual clar pentru toate acțiunile, cu etichete descriptive pentru formulare și butoane
- Interfață responsive funcțională pe rezoluții de minimum 1366x768
- Opțiune de temă clară și întunecată (light/dark mode) cu preferință persistentă
- Navigare consistentă pe toate paginile aplicației

3.3.2.5 Mentenabilitate și Portabilitate (CNF-42 – CNF-51)

- Cod sursă versionat folosind Git, cu convenții de stil consistente (ESLint pentru JavaScript, PEP8 pentru Python)
- Arhitectură modulară ce permite actualizarea individuală a componentelor
- Aplicația rulează în containere Docker pentru portabilitate maximă (exceptând serverul de simulare)
- Configurații specifice mediului gestionate prin variabile de mediu
- Sistem instalabil pe Linux, Windows și macOS (prin Docker)
- Interfață web compatibilă cu Chrome 90+, Firefox 88+, Edge 90+ și Safari 14+
- Suport pentru modele TensorFlow 2.x și PyTorch 2.x

3.4 Testare

Testarea platformei de simulare a atacurilor Data Poisoning acoperă multiple niveluri, de la validarea componentelor individuale până la testarea integrată a întregului flux de simulare. Dată fiind natura distribuită a arhitecturii (frontend containerizat, backend containerizat, orchestrator pe server remote cu GPU), strategia de testare se concentrează pe verificarea corectitudinii fiecărei etape din pipeline-ul de simulare, a comunicării între componente și a rezilienței sistemului la erori.

3.4.1 Testarea de Validare a Șabloanelor

Primul nivel de testare are loc înainte de consumarea oricăror resurse computaționale. Modulul `verify_template.py` realizează o **analiză statică** a scriptului de antrenare furnizat de utilizator, verificând:

- Existența tuturor funcțiilor obligatorii în șablon (funcția de descărcare date `download_data()`, funcția de creare model, funcțiile de setare și extragere a ponderilor)
- Conformitatea cu contractul de interfață așteptat de platforma de simulare
- Absența constructelor de cod potențial periculoase sau a importurilor nepermise

Această validare previne erorile tardive în pipeline, economisind timp și resurse GPU. În cazul în care verificarea eșuează, utilizatorul primește un mesaj descriptiv cu funcțiile lipsă sau structurile incorecte.

3.4.2 Testarea Pipeline-ului de Simulare

Pipeline-ul de simulare parcurge 14 etape secvențiale, fiecare cu mecanism propriu de tratare a erorilor. Testarea funcțională verifică:

- **Verificarea corectitudinii modelului (Etapa 4)** — după încărcarea șablonului, se execută o sesiune de antrenare de validare pe GPU-ul alocat. Acuratețea inițială este înregistrată în fișierul `init-verification.json`, servind drept referință pentru evaluări ulterioare
- **Descărcarea și partiționarea datelor (Etapa 6)** — se verifică descărcarea corectă a setului de date prin funcția `download_data()` și partiționarea în N split-uri IID egale (câte unul per client federat), cu setul de test partajat între toți clienții
- **Procesul de otrăvire a datelor (Etapa 9)** — se validează crearea corectă a copiei otrăvite pentru fiecare din cele 7 tipuri de atacuri suportate, verificând aplicarea parametrilor configurați (intensitate, procentaj, tip trigger)
- **Simulările federate (Etapele 10–13)** — se testează execuția corectă a celor patru scenarii (Clean, Clean+Defense, Poisoned, Poisoned+Defense), verificând că toate pornesc de la aceeași configurație inițială a modelului
- **Generarea rezultatelor (Etapa 14)** — se verifică crearea fișierelor `analysis.json`, `summary.txt` și `attack_info.json` cu metrici complete și corecte

La fiecare etapă, în cazul unei erori, pipeline-ul se oprește, generează un raport de eroare și transmite informația înapoi către utilizator prin WebSocket.

3.4.3 Testarea de Integrare

Testarea de integrare verifică fluxul complet de date între cele trei componente ale arhitecturii:

- **Frontend → Backend**: Trimiterea cererii POST cu șablonul Python și configurația simulării de la interfața React către API-ul FastAPI. Se verifică validarea de securitate a codului, stocarea configurației în PostgreSQL și retransmiterea asincronă către orchestrator
- **Backend → Orchestrator**: Comunicarea REST API între backend-ul containerizat și orchestratorul Flask de pe serverul ATM. Se testează autentificarea cu credențiale, transmiterea parametrilor și primirea status-urilor de progres
- **Orchestrator → Backend → Frontend**: Transmiterea rezultatelor finale de la orchestrator prin polling HTTP, colectarea fișierelor JSON cu metrici și afișarea lor în interfața utilizatorului

3.4.4 Testarea Comunicării WebSocket

Comunicarea în timp real este testată prin:

- Verificarea stabilirii conexiunii WebSocket între frontend și backend
- Testarea loop-ului asincron de polling (la fiecare 2 secunde) al stării simulării de la orchestrator
- Validarea transmiterii corecte a actualizărilor de progres (etapă curentă, status GPU, metrici intermediare
- Testarea mecanismului de reconectare automată în cazul întreruperii conexiunii

3.4.5 Testarea Managementului GPU

Componenta `GPUManager` este testată pentru:

- Interogarea corectă a stării GPU-urilor disponibile prin `nvidia-smi`
- Alocarea thread-safe a GPU-urilor din coada de resurse disponibile
- Respectarea timeout-ului de 10 minute și fallback-ul pe CPU
- Eliberarea garantată a GPU-ului (`finally` block) la finalizarea, anularea sau eșecul simulării
- Distribuirea echilibrată a simulărilor pe cele 3 GPU-uri NVIDIA A40 disponibile

3.4.6 Testarea Fluxului de Anulare

Mecanismul de anulare a simulărilor este testat prin:

- Verificarea interogării stării de anulare la fiecare etapă din pipeline
- Testarea terminării corecte a arborelui de procese (`kill_process_tree()`)
- Validarea eliberării GPU-ului alocat și ștergerii directorului de lucru
- Confirmarea setării corecte a statusului la *cancelled* și notificarea utilizatorului

3.4.7 Testarea de Securitate

Aspectele de securitate testate includ:

- Validarea token-urilor JWT: verificarea semnăturii digitale, expirarea și existența utilizatorului în baza de date pentru fiecare cerere protejată
- Validarea codului Python încărcat de utilizatori: detectarea constructelor periculoase și execuția în medii izolate (medii Anaconda separate)
- Izolarea datelor între utilizatori: verificarea că fiecare utilizator accesează exclusiv propriile proiecte, fișiere și rezultate
- Stocarea securizată a credențialelor prin Docker Secrets (parolă bază de date, cheie JWT, parolă orchestrator)
- Configurarea CORS pentru restricționarea accesului API doar la originile autorizate

3.5 Modele de Analiză

În această secțiune sunt prezentate modelele de analiză ale platformei, incluzând diagramele de context, componente, flux, entitate-relație și cazuri de utilizare. Aceste diagrame au fost elaborate în cadrul documentului de Specificații ale Cerințelor Software (SRS) și oferă o perspectivă vizuală asupra arhitecturii și funcționalităților platformei.

3.5.1 Diagrama de Context

Diagrama de context prezintă marginile platformei și entitățile externe care interacționează cu sistemul. La Nivelul 0, sunt identificați actorii principali: Cercetătorii (Data Scientists), Instituțiile și Administratorul, fiecare cu roluri specifice în cadrul platformei.

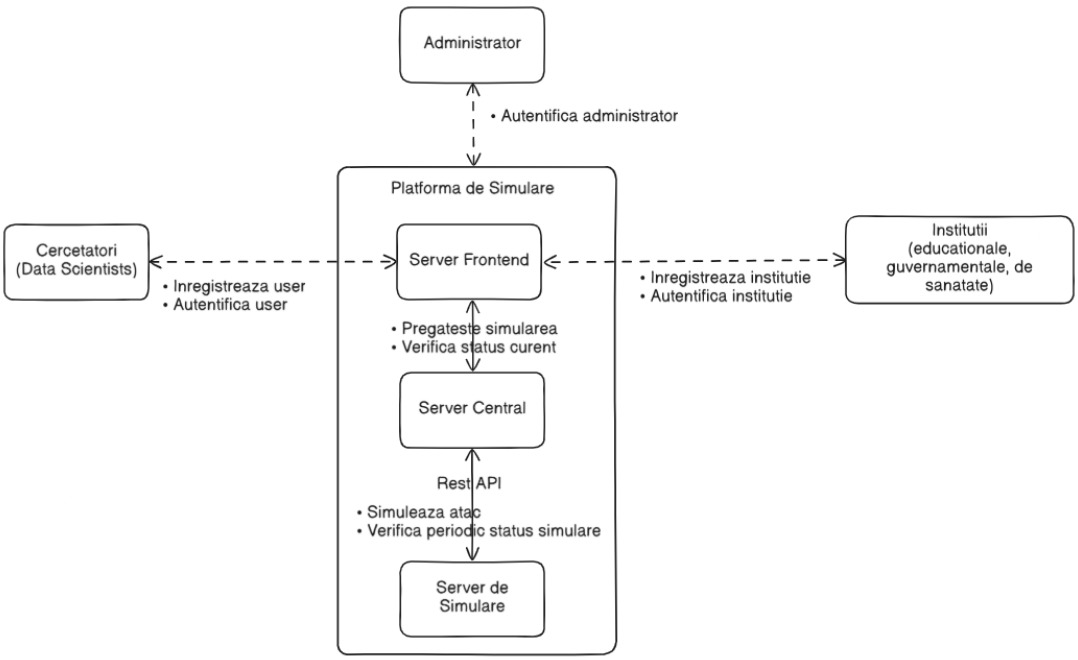

Figura 3.3: Diagrama de Context — Nivel 0: Marginile Platformei

La Nivelul 1, diagrama detaliază fluxul de date între sistem și entitățile externe, evidențiind interacțiunile dintre interfețele publice și protejate ale frontend-ului, serverul backend central cu baza de date PostgreSQL și serverul de simulări.

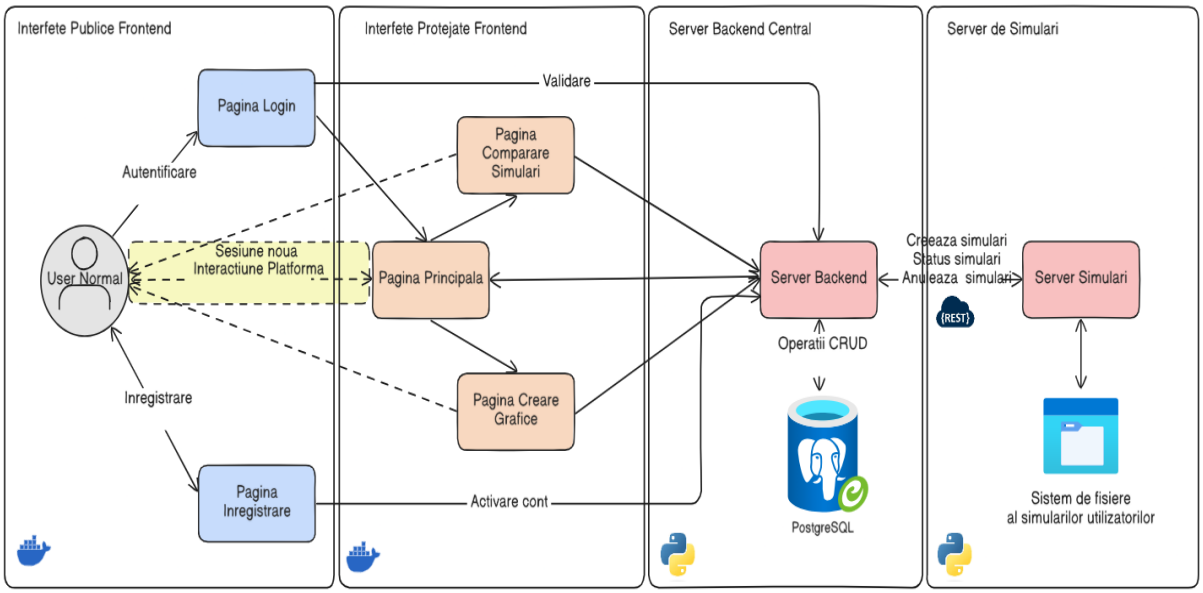

Figura 3.4: Diagrama de Context — Nivel 1: Fluxul de date

3.5.2 Diagrama de Componente

Diagrama de componente prezintă structura internă a platformei, evidențiind cele două servere principale (Server Central și Server de Simulări Remote) împreună cu componentele software asociate: React Frontend, PostgreSQL, Fast API pe serverul central, respectiv GPU Manager și Python Orchestrator pe serverul de simulări. De asemenea, sunt ilustrate fluxurile de date de la configurarea simulării până la generarea rezultatelor finale.

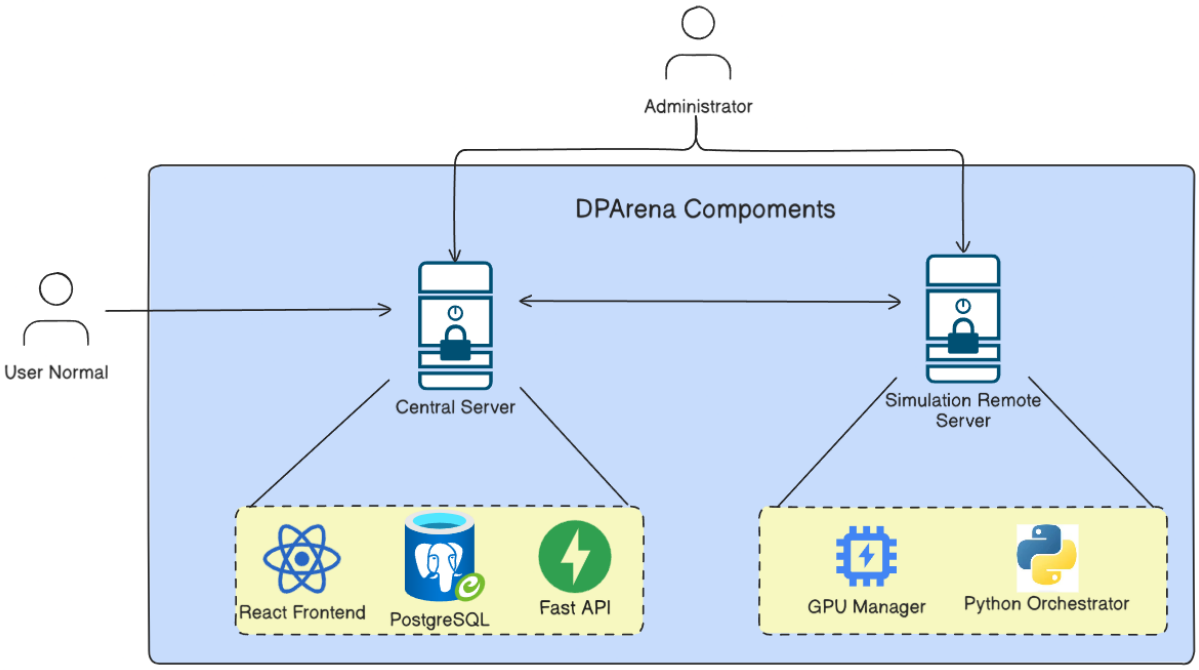

Figura 3.5: Diagrama de Componente a Platformei

3.5.3 Diagramele de Flux

Diagramele de flux prezintă procesele principale ale platformei din perspectiva utilizatorului și a simulării.

Diagrama fluxului utilizatorului ilustrează parcursul complet al unui utilizator în platformă, de la înregistrare și autentificare, prin integrarea multi-nivel (organizație, mediu de cercetare), până la accesul la dashboard și funcționalitățile principale: vizualizarea rezultatelor, crearea de simulări, compararea rezultatelor și gestionarea profilului.

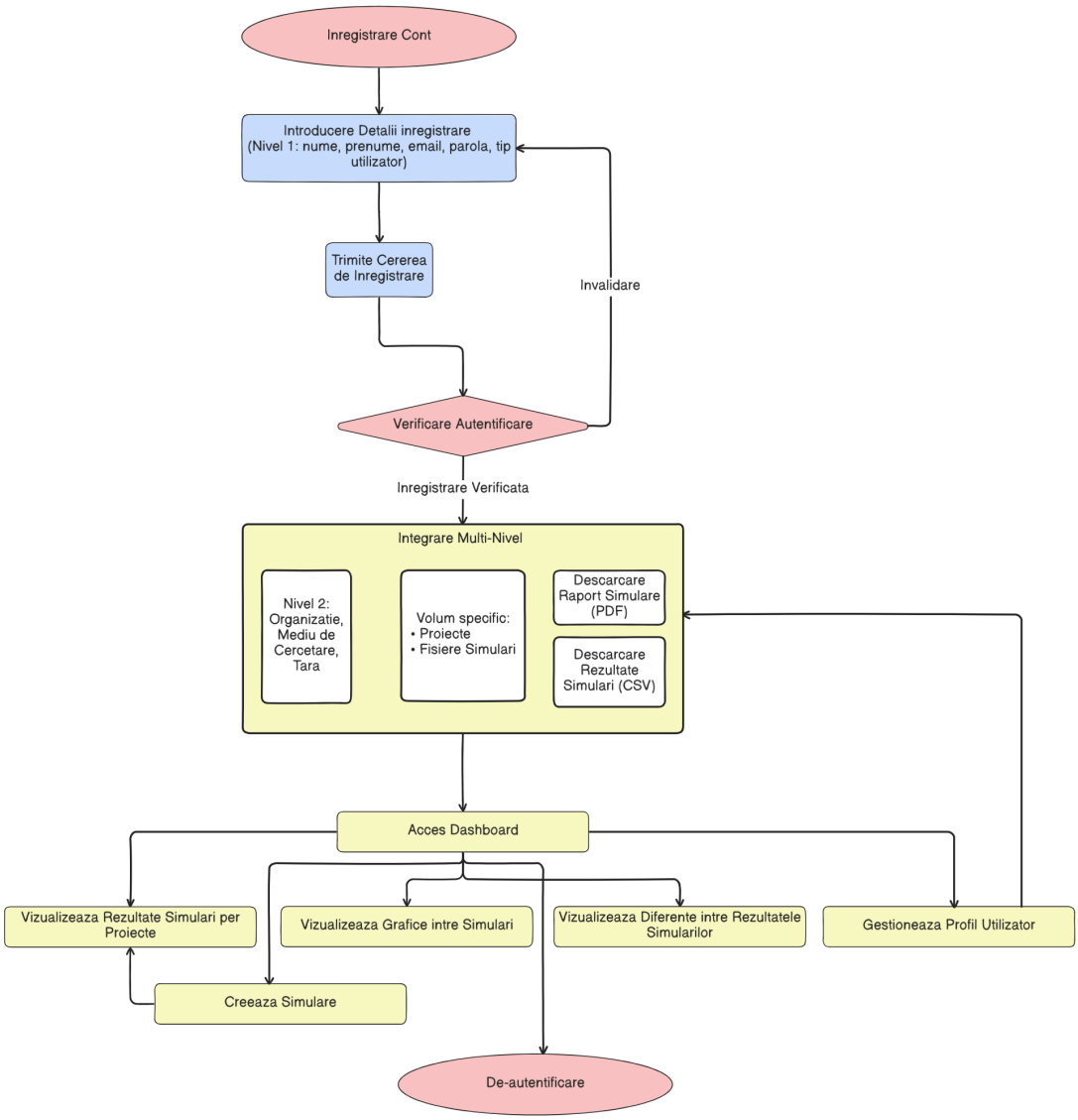

Figura 3.6: Diagrama Flux Utilizator

Diagrama fluxului de simulare detaliază etapele pipeline-ului de simulare, de la alocarea GPU și verificarea template-ului, prin descărcarea rețelei și a setului de date, aplicarea atacului de otrăvire, până la execuția celor trei scenarii de simulare federată (date curate, date otrăvite, date otrăvite cu funcție de agregare) și generarea rezultatelor finale.

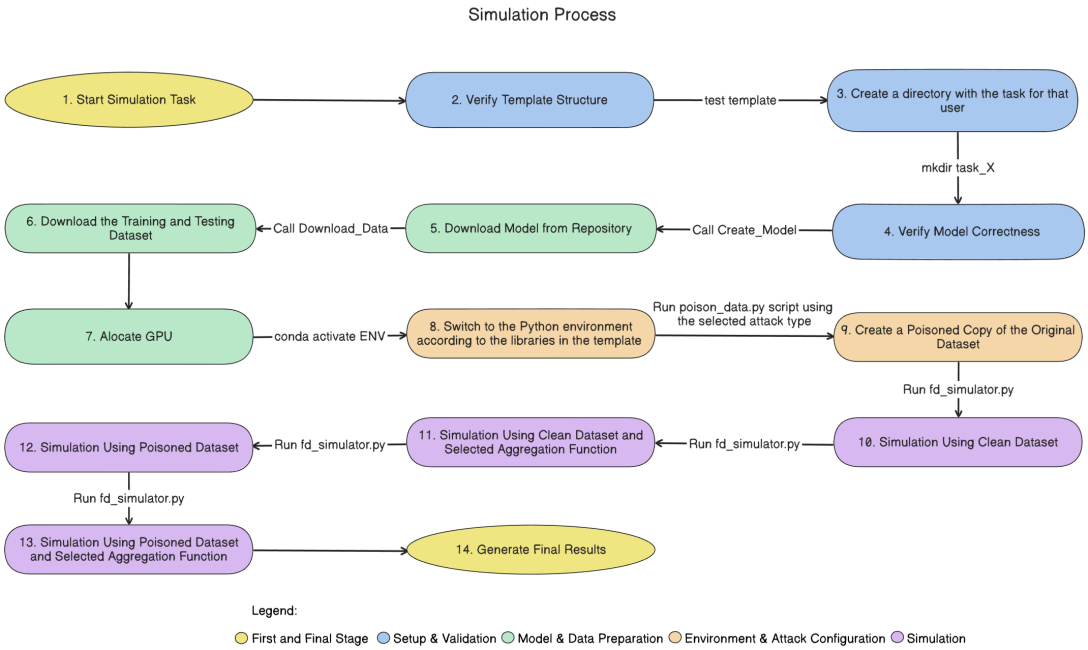

Figura 3.7: Diagrama Fluxului de Simulare

3.5.4 Diagrama Entitate-Relație (ERD)

Diagrama ERD prezintă structura bazei de date PostgreSQL a platformei, cu cele patru tabele principale: `users`, `projects`, `files` și `simulation_results`. Sunt evidențiate relațiile dintre entități: un utilizator creează mai multe proiecte, un proiect conține mai multe fișiere, iar rezultatele simulărilor sunt asociate cu utilizatorul, proiectul și fișierul corespunzător.

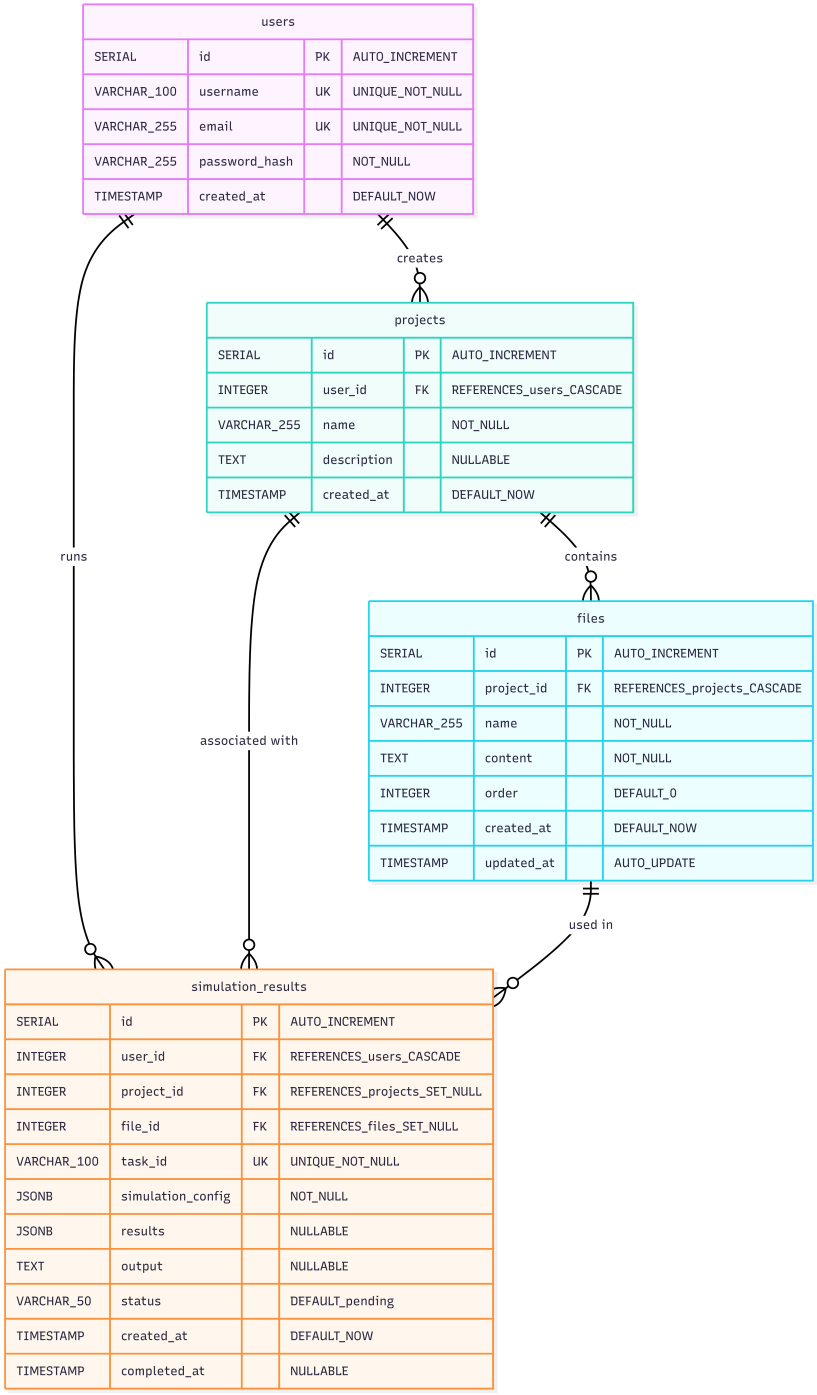

Figura 3.8: Diagrama Entitate-Relație (ERD)

3.5.5 Diagrama Cazurilor de Utilizare

Diagrama cazurilor de utilizare prezintă funcționalitățile platformei din perspectiva celor doi actori principali: Administratorul și Utilizatorul Normal. Administratorul dispune de capabilități de gestionare a conturilor, vizualizare alerte și jurnale de audit, iar utilizatorul normal accesează funcționalitățile de simulare, configurare, monitorizare și analiză a rezultatelor.

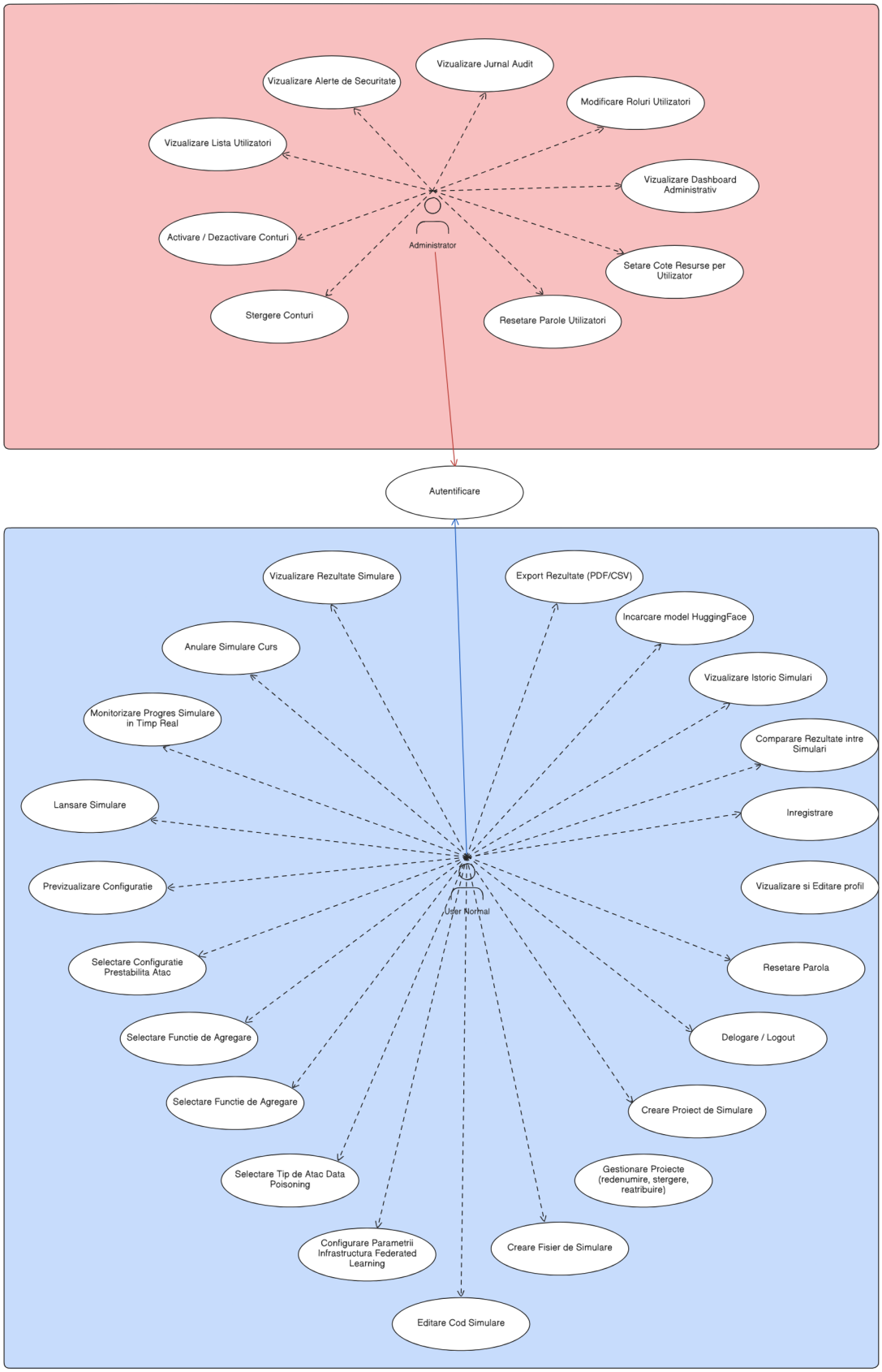

Figura 3.9: Diagrama Cazurilor de Utilizare

Capitolul 4:

Rezultate și Metrici Simulări

Acest capitol prezintă validarea experimentală a platformei. Sunt ilustrate două studii de caz principale care demonstrează modul în care sistemul poate fi utilizat pentru evaluarea mecanismelor de apărare împotriva atacurilor de tip Data Poisoning într-un mediu de învățare federată.

4.1 Studiu de Caz 1: Creșterea intensității otrăvirii datelor

Scopul primului experiment este de a evalua modul în care performanța modelului global se degradează pe măsură ce proporția datelor compromise din rețea crește.

4.1.1 Detalii practice

Pentru mediul de antrenare a fost utilizat setul de date CIFAR-10, cuprinzând 60.000 de imagini distribuite în 10 clase. Antrenarea a fost realizată folosind framework-ul PyTorch pe o arhitectură ResNet18 pre-antrenată. Setul de date a fost eșantionat Non-IID (date distribuite neidentic și independent) în mod egal pe toți clienții, fiecare având 80% imagini dintr-o clasă dominantă și 20% din restul claselor. Fiecare rundă de antrenare locală a utilizat optimizatorul Adam cu o rată de învățare mică (0.001) și funcția de pierdere de tip Categorical Cross-Entropy.

Simularea a fost executată în șapte instanțe distincte, incrementând progresiv proporția datelor compromise din setul total: 1%, 5%, 10%, 15% și până la 55%. Pentru nodurile compromise, a fost aplicată tehnica de otrăvire „Label-Flipping” (inversarea etichetelor). Pentru a apăra rețeaua împotriva acestui atac, orchestratorul a folosit metoda robustă de agregare *FoolsGold*, care analizează similaritatea cosinusului actualizărilor pentru a penaliza contribuitorii malițioși. Întregul mediu a fost accelerat de unități GPU pentru a gestiona eficient cerințele riguroase de calcul.

4.1.2 Rezultate experimentale

Efectuarea tuturor etapelor de simulare pentru un număr fix de 10 clienți ($N = 10$), fiecare mediu federat rulând pentru 5 runde a câte 3 epoci pe rundă, a durat aproximativ două ore, un timp puternic influențat de distribuția Non-IID a datelor.

Etapa curată (Clean) și etapa exclusiv atacată (Poisoned) au utilizat algoritmul standard de agregare FedAvg, în timp ce instanțele rulate cu apărare au implementat metoda FoolsGold. Așa cum a fost demonstrat și în literatura de specialitate, cu cât au fost prezenți mai puțini clienți infectați, cu atât eficacitatea protecției calculate prin similitudini a scăzut ușor, lucru confirmat în intervalul de 1%–15% date alterate, unde doar un singur client era țintit de sistem pentru infecție. Pe de altă parte, medierea standard prin FedAvg s-a dovedit masiv instabilă sub un atac cu distribuție mare a impactului (în special la 40% și 50% date otrăvite). Metoda de apărare FoolsGold a reușit cu succes să atenueze efectele atacului de tip label-flipping în intervalul dominant de 20%–55% date compromise din populație, demonstrând că modelul poate fi protejat în mod fiabil prin setarea unei metode de agregare defensive corecte alături de parametrii corespunzători.

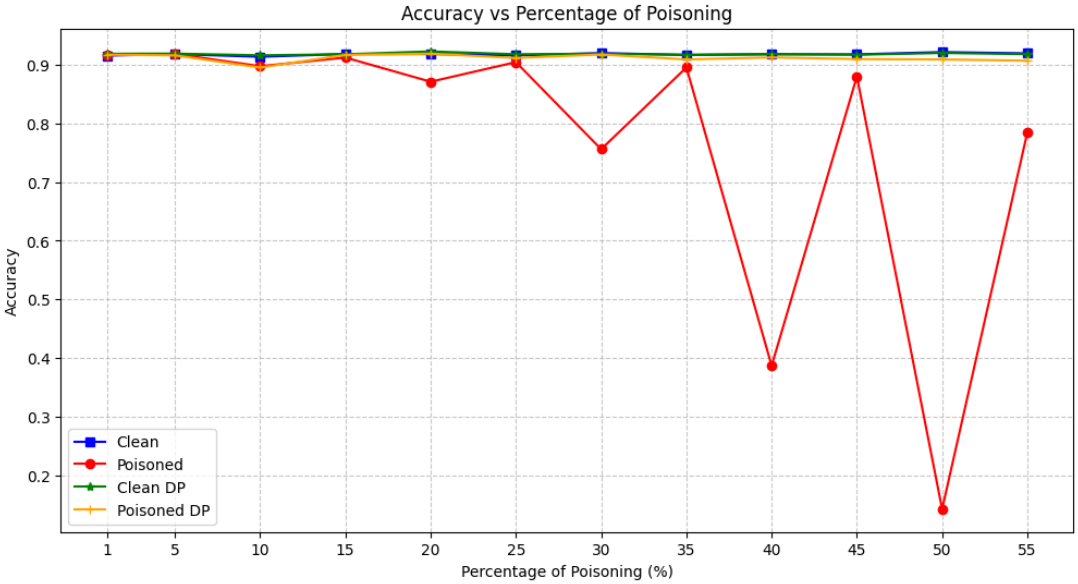

Figura 4.1: Rezultate Experimentale - Cazul 1

4.2 Studiu de Caz 2: Cea mai bună protecție pentru un atac specific

În acest studiu de caz se consideră un scenariu în care un cercetător dorește identificarea celui mai eficient mecanism de apărare pentru un atac fix de otrăvire a datelor. Obiectivul este de a compara mai multe strategii de protecție (agregare) în aceleași condiții experimentale. Spre deosebire de testul anterior, arhitectura modelului, tipul atacului și intensitatea rămân complet constante.

4.2.1 Detalii practice

Același model ResNet18 antrenat pe CIFAR-10 a fost folosit ca referință. Rețeaua federată a constat dintr-un total de 10 clienți interconectați ($N = 10$). Pentru a simula o prezență malițioasă extrem de dăunătoare și coordonată, 3 dintre acești clienți ($M = 3$, reprezentând 30% din rețea) au fost setați să acționeze sistematic împotriva orchestratorului. Antrenamentul s-a desfășurat pe durata a 5 runde globale, iar atacul de tip Label-Flipping a vizat 100% din datele de antrenament locale ale nodurilor aservite atacatorului.

În aceste condiții dure și fixe, au fost realizate mai multe simulări, singura variabilă modificată fiind metoda de apărare de la nivelul serverului central. Mai exact, a fost analizată performanța a patru mecanisme principale: FoolsGold, Trimmed Mean, Krum și Norm Clipping.

4.2.2 Rezultate experimentale

Evaluările au evidențiat diferențe majore în eficacitatea metodelor de apărare împotriva acestui grad ridicat de compromitere (30% din dispozitive):

Acuratețea modelului pre-antrenat inițial (descărcat înainte de propagare) a fost calculată la aproximativ 0.7334, iar performanța curată de referință a modelului antrenat federat (fără atac, folosind exclusiv agragarea FedAvg normală) a atins rapid pragul de 0.8556. Analizând

comportamentul sub tensiunea atacului de label-flipping, s-au obținut următoarele diferențe notabile:

- **FedAvg (fără protecție)** a servit drept variantă de subsol (baseline sub atac) și a generat o acuratețe finală de doar 0.5546. Aceasta reprezintă o scădere fatală de -0.1788 chiar și față de modelul inițial descărcat de pe internet și o scădere de -0.3010 față de media curată optimă, confirmând vulnerabilitatea critică a medierii ponderate clasice în fața unui atac coordonat din partea a o treime din rețea.
- **FoolsGold** s-a dovedit a fi clar cea mai eficientă metodă defensivă pe acest tip de configuratie, atingând o acuratețe de 0.8464 în ciuda celor 3 agenți atacatori activi — o scădere microscopică de numai -0.0092 față de situația curată fără niciun atac. Analizând actualizările prin similaritate cosinus, orchestratorul a putut depista că toți clienții atacatori furnizează un set comun de caracteristici greșite și le-a redus drastic sau chiar suprimat aportul spre modelul global.
- **Norm Clipping** a obținut al doilea cel mai bun rezultat, cu o acuratețe finală menținută la nivelul de 0.7468 (reprezentând o creștere reziduală de $+0.0134$ comparativ cu modelul pre-antrenat inițial). Limitând magnitudinea permisă pentru norma vectorilor trimiși de clienți, mecanismul a oprit creșterea forțată a ponderilor infectate dinspre atacatori fără a fi nevoit să își reducă rețeaua total acceptată de conexiuni.
- **Trimmed Mean** a restrâns o acuratețe moderată la nivelul de 0.5874. Deși a adus o îmbunătățire utilă față de absența totală a apărării, eliminarea parametrilor statistici de la extemele superioare și inferioare nu a asigurat o barieră completă, efortul coordonat creand un grad mare de raportări eronate.
- **Krum** a eșuat și a oferit paradoxal cea mai slabă protecție din cadrul bateriei de teste, obținând o acuratețe de doar 0.5132 (situată chiar sub rezistența FedAvg simplu). Concepția logicii acestui algoritm alege un singur vector din rândul update-urilor clienților preluându-l pe cel ce are cea mai mică distanță euclidiană față de ceilalți. Cei 3 clienți malițioși cu pachete foarte similare, dar profund eronate s-au susținut teoretic reciproc și au ridicat exponențial riscul algoritmului de a califica doar o „încercare de atac” drept candidat valid de încredere, corupând în final total mediul.

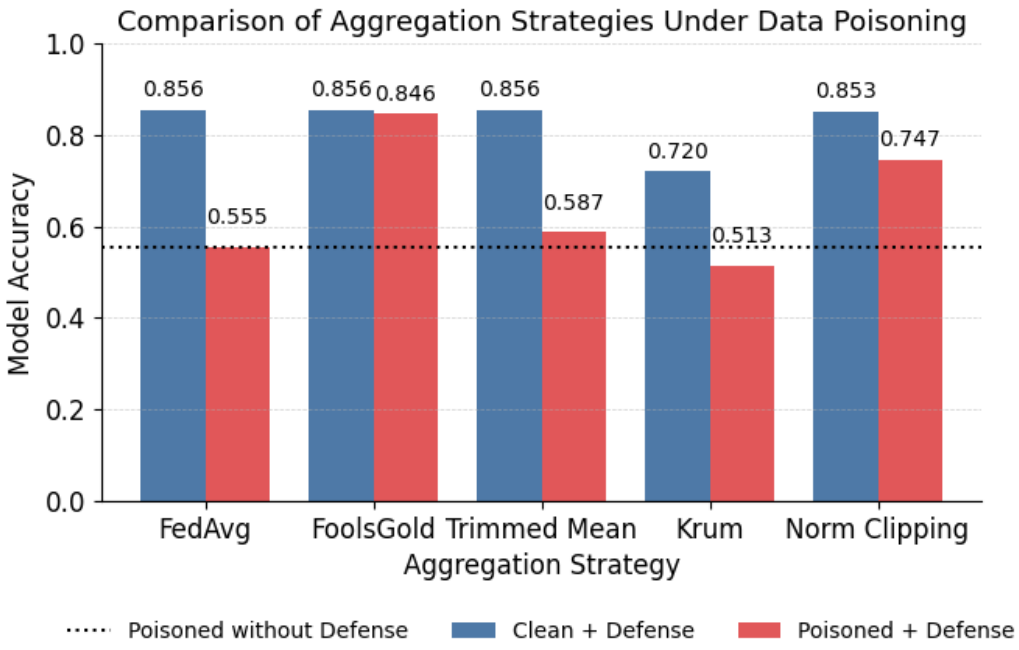

Figura 4.2: Rezultate Experimentale - Cazul 2

Capitolul 5:

Concluzii si dezvoltare ulterioara

Platforma prezenta acestei lucrari este o provocare in domeniul securitatii algoritmilor de invatare automata prin crearea unui mediu izolat specific pentru testarea atacului de otravire a datelor. Aceasta a fost construita pentru a ajuta atat inginerii de securitate cibernetica, precum si inginerii de date care lucreaza cu retele neuronale deja antrenate, sa realizeze o baterie de teste cuprinzatoare inainte de a introduce in productie, intr-un mediu federat, in vederea imbunatatirii rezilientei retelei la acest tip de atac.

5.1 Starea Curenta

Proiectul prezentat in starea curenta, este format din doua parti. Prima parte, cea in care se construieste mediul de containere Docker si a doua, in care se construieste orchestratorul si se implementeaza etapele din jurul lui, prezentate in Figura 3.2. In implementarea actuala, platforma dispune de o interfata moderna, in care utilizatorul se inregistreaza si isi poate crea mai multe proiecte si teste (simulari) cu diferite moduri de configurare. La finalizarea fiecarei simulari, rezultatul este vizibil si poate fi salvat local sau poate fi comparat cu alte simulari. Platforma suporta antrenarea de retele neuronale folosind biblioteca tensorflow si permite realizarea tuturor etapelor prezentate in capitolul 3.

5.2 Dezvoltare Ulterioara

In procesul ulterior de dezvoltare, se vor integra si bibliotecile de pytorch pentru o gama completa de biblioteci de antrenare si se va oferi posibilitatea utilizatorului sa isi aleaga tipul preferat. Pe langa acest aspect, se va lua in considerare si ajustarea tipului de atac Gaussian Noise folosind un tip de noise compus din ponderile primite de la dispozitivele asociate.

O privire de ansamblu asupra problemelor deschise, limitărilor cunoscute și riscurilor tehnice identificate pentru platforma de simulare este documentată în Anexa B — Lista Problemelor din cadrul documentului de Specificații ale Cerințelor Software (SRS). Printre aspectele semnalate se numără: dimensiunea maximă acceptată pentru fișierele de model, limita de 100 de clienți federați impusă de constrângerile de memorie GPU, suportul exclusiv pentru seturi de date de tip imagine, dependența de disponibilitatea serviciilor externe (HuggingFace Hub) și riscul pierderii conexiunii WebSocket în timpul simulărilor de lungă durată. Aceste aspecte vor fi adresate prioritar în iterațiile viitoare ale platformei.

Bibliografie

Cărți

Articole Științifice

[6] Peter Kairouz et al., "Advances and Open Problems in Federated Learning", în *Foundations and Trends® in Machine Learning* 14.1-2 (2021), pp. 1–210, DOI: 10.1561/2200000083, URL: https://doi.org/10.1561/2200000083.

[7] Keith Bonawitz, Hubert Eichner și Wolfgang Grieskamp, "Towards Federated Learning at Scale: System Design", în *Proceedings of the 2nd SysML Conference*, Palo Alto, CA, USA, 2019.

Referințe Online și Diverse

[1] Directoratul Național de Securitate Cibernetică (DNSC), *Buletin de Indicatori Statistici și Tendințe de Securitate Cibernetică - H1 2025*, https://www.dnsc.ro/vezi/document/buletin-de-indicatori-statistici-si-tendinte-de-securitate-cibernetica-h1-2025, Accesat în 2025-11-27, 2025.

[2] Future of Life Institute, *AI Act Overview - Future of Life Institute (30 May 2024)*, https://artificialintelligenceact.eu/wp-content/uploads/2024/11/Future-of-Life-InstituteAI-Act-overview-30-May-2024.pdf, Accessed: 2025-11-27, mai 2024.

[3] IBM, *IBM Federated Learning API Documentation*, https://ibmfl-api-docs.res.ibm.com/index.html, Accesat în 2025-11-28, 2025.

[4] IBM, *IBM Federated Learning Library - GitHub Repository*, https://github.com/IBM/federated-learning-lib/tree/main, Accesat în 2025-11-28, 2025.

[5] IBM, *AI vs. Machine Learning vs. Deep Learning vs. Neural Networks*, https://www.ibm.com/think/topics/ai-vs-machine-learning-vs-deep-learning-vs-neural-networks, Accesat în 2025-11-28, 2025.

[8] Clement Fung, Chris J. M. Yoon și Ivan Beschastnikh, *Mitigating Sybils in Federated Learning Poisoning*, 2020, arXiv: 1808.04866 [cs.LG].